Mississippi State University
Scholars Junction

Theses and Dissertations | Theses and Dissertations

5-16-2025

Bucket-based priority queues for A* and related bounded-suboptimal and anytime search algorithms: Theoretical and practical advancements

Garrett Michael Fereday
Mississippi State University, gmfereday@gmail.com

Follow this and additional works at: https://scholarsjunction.msstate.edu/td

Part of the Computer Sciences Commons

Recommended Citation

Fereday, Garrett Michael, "Bucket-based priority queues for A* and related bounded-suboptimal and anytime search algorithms: Theoretical and practical advancements" (2025). *Theses and Dissertations*. 6484.
https://scholarsjunction.msstate.edu/td/6484

This Dissertation - Open Access is brought to you for free and open access by the Theses and Dissertations at Scholars Junction. It has been accepted for inclusion in Theses and Dissertations by an authorized administrator of Scholars Junction. For more information, please contact sct@library.msstate.edu.

Bucket-based priority queues for A* and related bounded-suboptimal and anytime search algorithms: Theoretical and practical advancements

This dissertation - open access is available at Scholars Junction: https://scholarsjunction.msstate.edu/td/6484

Bucket-based priority queues for A* and related bounded-suboptimal and anytime search algorithms: Theoretical and practical advancements

By

Garrett Michael Fereday

Approved by:

Eric A. Hansen (Major Professor)
Jingdao Chen
Maxwell Young
Rong Zhou
T.J. Jankun-Kelly (Graduate Coordinator)
David M. Ford (Dean, College of Engineering)

A Dissertation
Submitted to the Faculty of
Mississippi State University
in Partial Fulfillment of the Requirements
for the Degree of Doctor of Philosophy
in Computer Science
in the Department of Computer Science and Engineering

Mississippi State, Mississippi

May 2025

Copyright by

Garrett Michael Fereday

2025

Name: Garrett Michael Fereday

Date of Degree: May 16, 2025

Institution: Mississippi State University

Major Field: Computer Science

Major Professor: Eric A. Hansen

Title of Study: Bucket-based priority queues for A* and related bounded-suboptimal and anytime search algorithms: Theoretical and practical advancements

Pages of Study: 101

Candidate for Degree of Doctor of Philosophy

For shortest-path problems with a small number of integer transition costs, it is well-known that the performance of the classic A* algorithm can be improved by using *bucketing* to reduce priority queue overhead—in particular, by using a bucket queue data structure for the priority queue, instead of a binary heap. This dissertation describes several theoretical and practical extensions of this approach.

First, the traditional two-level bucket queue data structure is modified in simple ways to improve the worst-case complexity of its operations, which leads to the first demonstration that the priority queue operations of a two-level bucket queue for A* can have amortized constant-time complexity. Second, and more importantly, a generalization of the bucket queue data structure is introduced, called a bucket heap, that can be used by a large class of bounded-suboptimal and anytime variants of the A* algorithm. Analysis shows that the operations of a bucket heap have improved asymptotic complexity compared to a binary heap. In addition, experiments involving a wide range of search

algorithms and domains show that use of a bucket heap leads to faster search and improved suboptimality bounds.

Finally, a hybrid algorithmic strategy is proposed that bounds the worst-case number of state re-expansions performed by bounded-suboptimal and anytime search algorithms, which expand states in order of an inadmissible evaluation function. This strategy is most efficiently implemented by using a bucket heap and it guarantees that the number of expansions performed by the algorithm is asymptotically equivalent to the number of expansions performed by A*. Experiments also show that this hybrid approach can lead to further speedup for many search problems, as well as improved suboptimality bounds.

Key words: heuristic search, bounded-suboptimal search, anytime search, priority queue, bucket queue, bucket heap

DEDICATION

This dissertation is dedicated to my wife Isidora, who has been my greatest supporter during this long and arduous process. I couldn't have done it without you!

TABLE OF CONTENTS

DEDICATION ii

LIST OF TABLES v

LIST OF FIGURES vii

CHAPTER

1. INTRODUCTION 1

2. BACKGROUND 3

2.1 State-Space Representation 3
2.2 Uniform Cost Search and A* 5
2.3 Bounded-Suboptimal Search 9
2.4 Anytime Search 10
2.5 Priority Queue Data Structures 11
2.6 Binary Heap 13
2.6.1 Operations and Complexity Analysis 14
2.7 Bucket Queue 19
2.7.1 Operations and Complexity Analysis 21

3. TWO-LEVEL BUCKET QUEUE FOR A*: ANALYSIS AND IMPROVEMENTS 25

3.1 Two-Level Bucket Queue for A* 25
3.1.1 Operations and Complexity Analysis 27
3.2 Modified Two-Level Bucket Queue with Improved Worst-Case Complexity 32
3.2.1 Complexity Analysis 33
3.3 Lazy Two-Level Bucket Queue for A* 36
3.3.1 Experimental Comparison 38
3.4 Exploiting Locality of Reference 41
3.4.1 Experimental Comparison 43

4. THE BUCKET HEAP 47

4.1 Structure 47
4.2 Complexity Analysis 48
4.2.1 Enqueue 50
4.2.2 Dequeue 51
4.2.3 ChangeKey 52
4.2.4 LowerBound and Reorder 53
4.2.5 Memory 55
4.3 Related Work 55
4.3.1 Structure and Operation Complexities 56
4.4 Theoretical Comparison 57
4.5 Experiments 60
4.5.1 Test Algorithms 60
4.5.2 Test Domains 63
4.5.3 Data analysis 66

5. ALGORITHMIC IMPROVEMENTS ENABLED BY THE BUCKET HEAP . . 78

5.1 Computing a Better Suboptimality Bound 78
5.2 Limiting State Re-Expansions 82
5.3 Achieving Tighter Suboptimality Bounds 87

6. CONCLUSION AND FUTURE RESEARCH 94

6.1 Future Research 95

REFERENCES 99

LIST OF TABLES

3.1 The best-case and worst-case complexities of operations for each of the priority queues. 34

3.2 The speedup factor of using a dynamic array vs. a linked list as the secondary buckets in the two-level bucket queue. 40

3.3 The speedup factor of each two-level bucket queue variant relative to the standard two-level bucket queue. 41

3.4 The speedup factor of hash table operations using the h-value indexed array of hash tables vs. a single hash table. 46

4.1 The worst-case complexities of operations for each of the priority queues. 57

4.2 The complexities of operations on the binary heaps of buckets for the bucket heap and the Gilon structure. $\hat{N}_p$ is the number of non-empty primary buckets, $\hat{N}_s$ is the number of non-empty secondary buckets. 59

4.3 The speedup factor of priority queue operations for each algorithm relative to their implementations using a binary heap (or two, where necessary), averaged across Korf's 100 15-puzzle instances. 72

4.4 The speedup factor of priority queue operations for each algorithm relative to their implementations using a binary heap (or two, where necessary) in the 6-DoF unit-cost robot arm domain. 72

4.5 The speedup factor of priority queue operations for each algorithm relative to their implementations using a binary heap (or two, where necessary) in the 6-DoF non-uniform cost robot arm domain. 72

4.6 The speedup factor of priority queue operations for OPS relative to the implementation using two binary heaps in two-dimensional grid navigation. 76

4.7 The speedup factor of priority queue operations for BAWA* relative to the implementation using two binary heaps in two-dimensional grid navigation. 76

4.8 The speedup factor of priority queue operations for DPS relative to the implementation using two binary heaps in two-dimensional grid navigation. 76

4.9 The speedup factor of priority queue operations for ANA* relative to the implementation using a binary heap in two-dimensional grid navigation. 76

4.10 The speedup factor of priority queue operations for OPS relative to the implementation using two binary heaps in multiple sequence alignment. 77

4.11 The speedup factor of priority queue operations for BAWA* relative to the implementation using two binary heaps in multiple sequence alignment. 77

4.12 The speedup factor of priority queue operations for DPS relative to the implementation using two binary heaps in multiple sequence alignment. 77

4.13 The speedup factor of priority queue operations for ANA* relative to the implementation using a binary heap in multiple sequence alignment. 77

5.1 Expansion ratios relative to A* for the 48-pancake problem. 86

5.2 Expansion ratios relative to A* for the different sizes of two-dimensional grid. . . . 87

5.3 Suboptimality bounds and solution costs achieved in 5 million expansions for the heavy 16-pancake problem. 90

5.4 The number of instances solved for various suboptimality bounds on the heavy 16-pancake problem. 93

LIST OF FIGURES

2.1 A binary heap of integers, where an integer's value is its priority. 13

2.2 A bucket queue of arbitrary states, where $T = 4$. 21

3.1 (a) A high-level view of an arbitrary two-level bucket queue, omitting the third dimension. Shaded cells indicate non-empty secondary buckets. (b) An expanded view of the f_{min} primary bucket. The nodes A–F represent arbitrary states. 26

3.2 An example where the worst-case complexity of UpdateMin occurs in the two-level queue. S_0 indicates the initial state, fully-shaded cells mark impassable regions, cross-hatched cells mark expanded states, and the states on the OPEN list are shaded, with their f-values labeled. 29

3.3 (a) The initial configuration for instance 1 of Korf's 100 puzzles. (b) The desired goal configuration. 39

3.4 The initial configuration for the 6-DoF robot arm problem. Shaded cells represent obstacles in the environment and the goal location is marked by the letter 'G'. . . . 45

4.1 An example alignment of eight DNA sequences [25]. 66

5.1 The suboptimality bounds per expansion for ANA* and Focal ANA* on instance 100 of the 15-puzzle. 80

5.2 The suboptimality bounds per expansion for ANA* and Focal ANA* on the non-uniform-cost 6-DoF robot arm. 80

5.3 The suboptimality bounds per expansion for ANA*, Focal ANA*, and $\hat{f}_w(s)$ on instance 100 of the 15-puzzle. 83

5.4 The suboptimality bounds per expansion for ANA*, Focal ANA*, and $\hat{f}_w(s)$ on the non-uniform-cost 6-DoF robot arm. 83

5.5 The suboptimality bounds per expansion for Focal ANA*, $\hat{f}_w(s)$, and their hybrid implementations on instance 100 of the 15-puzzle. 88

5.6 The suboptimality bounds per expansion for Focal ANA*, $\hat{f}_w(s)$, and their hybrid implementations on the non-uniform-cost 6-DoF robot arm. 88

5.7 The suboptimality bounds per expansion on an instance of the heavy 16-pancake problem. 91

5.8 The suboptimality bounds per expansion on the non-uniform-cost 20-DoF robot arm. 92

CHAPTER 1

INTRODUCTION

The concept of bucketing (or binning, or grouping) objects that have similar attributes is frequently used to accelerate algorithms in domains that are otherwise intractable; to significantly reduce computational overhead at the cost of (usually) minor loss in accuracy; or to provide asymptotic improvements by leveraging the specific characteristics of some domains. For example, when processing continuous data in two and three dimensions, quad-trees and oct-trees are used to discretize the data into regions that exhibit spatial locality [17]. Similarly, boundary volume hierarchies spatially group the data of triangular meshes in complex three dimensional scenes, dramatically reducing the cost of rendering these scenes with global illumination algorithms such as Monte Carlo path tracing [22]. Perhaps the most well-known use of bucketing is the bucket sort algorithm. When the keys to be sorted are (approximately) uniformly distributed, and the number of buckets used is proportional to the size of the array being sorted, bucket sort is capable of linear-time sorting [5].

As dynamic programming algorithms, the algorithms used to solve shortest-path problems in a graph do so incrementally, exploring first the paths of length 1, then the paths of length 2, and so on until the shortest path is discovered. These paths are explored in sorted order of length, and determining exploration order is then a modified version of the sorting problem where the

set being sorted is mutable, with elements being added or removed during the search. As with the sorting problem, it's possible to accelerate the process of selecting the next path to explore by using bucketing. A bucket-based priority queue was first introduced for Dijkstra's algorithm, and the structure was subsequently extended to the A* algorithm. However, for bounded-suboptimal and anytime search algorithms, the bucketing strategy is not as simple to apply, and it has not been shown how to generally apply bucket-based priority queues for these classes of algorithms.

The research presented in this dissertation has three major focuses: bucket-based priority queues for A*, generalization of bucket-based priority queues to the classes of bounded-suboptimal and anytime search algorithms, and introduction of a hybrid algorithmic strategy that leverages this generalization of bucket-based priority queues. After providing background information in Chapter 2, Chapter 3 addresses an existing gap in the literature by providing a detailed asymptotic analysis of the traditional two-level bucket queue for A*. Improvements to this data structure are then presented, culminating in the first two-level bucket queue for A* that provides both tie-breaking and amortized constant complexity.

In Chapter 4, a novel bucket-based priority queue is introduced that extends beyond A* to related bounded-suboptimal and anytime search algorithms. This data structure is shown to achieve an asymptotic complexity superior to that of a binary heap. Experimental analysis shows that it provides search speedup and improved suboptimality bounds for multiple algorithms in several search domains. Chapter 5 then presents a hybrid search strategy that bounds the number of state re-expansions performed by bounded-suboptimal and anytime search algorithms that use inadmissible evaluation functions. This hybrid approach is also shown to further improve the suboptimality bounds of the algorithms to which it is applied.

CHAPTER 2

BACKGROUND

This chapter provides background on state-space representation, foundational graph search algorithms, and terminology used throughout the rest of the dissertation. Multiple classes of heuristic graph-search algorithms are defined, and their differences are discussed. Two traditional priority queue data structures are defined, providing necessary knowledge for the rest of the dissertation.

2.1 State-Space Representation

State-space search is a long-standing branch of artificial intelligence and many different problems can be modeled within this framework. At the most fundamental level, a *state space* can be represented as a tuple $P = (S, A, c)$, where

- S is the set of all states
- A is the finite set of actions $\{a_1, ..., a_n\}$, such that $a_i : S \rightarrow S$ transforms one state to another
- c is a cost function $c : A \rightarrow \mathbb{R}$

Equivalently, a state-space can be represented as a weighted graph $G = (N, E, c)$ where

- $N = S$ is the set of nodes, n
- E is the set of edges, (n, n')
- c is a cost function $c : E \rightarrow \mathbb{R}$.

This dissertation will use the state and action nomenclature, but all discussions can be equivalently framed in terms of the graph representation.

In many domains, the size of a state space $|S|$ exceeds the capacity of computer memory. In these cases it is necessary to dynamically generate relevant regions of the state space during the search. In other words, the state space as a whole is represented *implicitly*, and new states are generated by applying a transformation to a state that is already in memory. Only generated states must be stored in memory; these states are referred to as the *explicit* graph. The process of generating the successors of a state is called *expanding* the state.

A specific *instance* of a state-space problem provides additional information and is described by a tuple $I = (S, A, s_0, S_T, c)$, where

- S, A, and c have the same definitions as above
- $s_0 \in S$ is the initial, or start state
- $S_T \subseteq S$ is the set of terminal, or goal states.

A *solution* $\pi = (a_1, ..., a_k)$ of a problem instance is an ordered set of actions that, when applied in sequence, transforms the initial state s_0 into a goal state $t \in S_T$. The *length* of a solution, $|\pi|$, is the number of actions taken to reach the goal state, and the total cost of a solution, $c(\pi)$, is the sum of the costs of its actions:

$$c(\pi) = \sum_{i=1}^{k} c(a_i).$$

In the special case of a uniform cost function, the cost of a solution is $c(\pi) = |\pi| \times u$, where u is the uniform cost. An *optimal solution* π^*, has the minimal total cost of all possible solutions. In

the case of a uniform cost function, it is clear that the optimal solution is the one with minimal length. More generally,

$$\pi^* = \min_{\pi} c(\pi).$$

The objective then, is to find a solution to the problem with optimal cost, $c(\pi^*)$, denoted as C^* from here on.

2.2 Uniform Cost Search and A*

Best-first search is a generic algorithmic approach to finding a minimum-cost path between states in a state-space graph. The key operation is determining which state should be expanded next, and best-first search algorithms do this by using a state *evaluation function*, $v(s) : s \rightarrow \mathbb{R}$. This function maps states to priority values, allowing them to be ordered for expansion by these priorities via a priority queue, referred to as the *OPEN* list. For the sake of cohesiveness, discussion of specific priority queue operations and implementations does not begin until Section 2.5. To avoid the need to reference these details, manipulation of the OPEN list will be shown in set notation.

The OPEN list maintains all states that have been generated but not yet expanded. When a state is expanded it is added to the *CLOSED* list, which stores all expanded states. If a goal state is expanded, then the search is terminated, as an optimal path has been found. Because there can be multiple paths to the same state in the graph, and not all of those paths are minimal, it is necessary to check whether a newly generated state has been previously expanded by checking if it exists within the CLOSED list. If the state exists in the CLOSED list and the new path is more expensive than the old path, then there is no need to insert it into the OPEN list and it can be discarded.

Algorithm 2.1: General pseudocode for best-first search.

```
Input: A, the set of actions
       s_0, the initial state
       S_T, the set of goal states
       c, the cost function
OPEN ← {s_0}
CLOSED ← ∅
while OPEN ≠ ∅ do
    s ← min_{s∈OPEN} v(s)
    OPEN ← OPEN \ {s}
    CLOSED ← CLOSED ∪ {s}
    if s ∈ S_T then
        return s
    foreach (s′, c_a) ∈ {(a(s), c(a)) | a ∈ A} do
        if s′ ∉ OPEN ∪ CLOSED then
            g(s′) ← g(s) + c_a
            OPEN ← OPEN ∪ {s′}
        else if g(s′) > g(s) + c_a then
            if s′ ∈ OPEN then
                OPEN ← OPEN \ {s′}
            else
                CLOSED ← CLOSED \ {s′}
            g(s′) ← g(s) + c_a
            OPEN ← OPEN ∪ {s′}
return nil
```

However, if the new path is *less* expensive, then the state will be removed from the CLOSED list and reinserted into the OPEN list with the new priority. Additionally, a similar scenario arises if a newly generated state already exists within the OPEN list. As before, if the new path is more expensive, then the duplicate state can be discarded. Alternatively, if the new path is less expensive, the state's priority in the OPEN list must be updated. Algorithm 2.1 gives pseudocode for best-first search as a general framework.

The key differentiator between best-first search algorithms is how the state evaluation function is defined. This function affects expansion order, state re-expansions, and overall runtime performance. The simplest evaluation function will expand states based on order of generation, and can be achieved by using a FIFO queue. This is simply breadth-first search, and it is clear that this is a special case of best-first search. However, breadth-first search will only provide optimal solutions in the case of a uniform cost function. Redefining the evaluation function to $v(s) = g(s)$, where $g(s)$ is the cost from s_0 to s, will allow for expanding nodes in minimal cost order by using a priority queue. When the cost function is uniform this is equivalent to breadth-first search, but for a non-uniform cost function this evaluation function still guarantees finding an optimal solution, should one exist. This is the evaluation function used by *Dijkstra's* algorithm and *Uniform Cost Search* (UCS) [7]. The primary difference between these two algorithms is that Dijkstra's is defined in the literature as operating on an explicit graph, whereas UCS operates on implicit graphs.

Dijkstra's algorithm and UCS are *uninformed* search techniques; that is, they do not make use of information about the specific search problem to which they are applied. While this does not prevent them from finding optimal solutions, utilizing problem-specific information may allow for a better expansion order, reducing the total number of state expansions, improving runtime. The

$A*$ algorithm is the first *informed* search technique in the literature and is capable of leveraging problem-specific information to improve its expansion order [13]. It does so by adding a heuristic estimate $h(s)$ to its evaluation function. This heuristic is an estimate of the *remaining* cost from a state to the goal, and the modified evaluation function used by A* is then $f(s) = g(s) + h(s)$, where $h(s) = 0, \forall s \in S_T$. The heuristic is domain dependent and aids in determining the expansion order, but there may be several viable heuristics for any domain. If the heuristic is an oracle that always provides the exact remaining cost to reach the goal, then A* will only expand nodes that lie on the optimal solution path. However, having a perfect heuristic is rare in practice, and the performance of A* in general depends heavily on the selected heuristic. When combined with the heuristic $h(s) = 0, \forall s \in S$, A* will have the same expansion order as UCS; because of this, UCS can be considered a special case of A*.

The heuristic estimate used in A*'s evaluation function can have a profound impact on the algorithm's performance, for better or worse. Because the heuristic must be computed for every generated state, there is a trade-off between the quality of the heuristic and the computation overhead that it incurs. While this is a trade-off that must be considered, the most crucial aspects of a heuristic are its *admissibility* and *consistency*. An admissible heuristic is one that will never *over-estimate* the remaining cost to the goal. Should it be possible for a heuristic to over-estimate this cost, the heuristic is considered to be *inadmissible*. In the presence of an inadmissible heuristic, A* is no longer guaranteed to find an optimal solution because a state on the optimal path may receive an inflated heuristic estimate and, subsequently, be ignored for expansion.

A consistent heuristic ensures that the estimate of a state is at most equal to the estimate of its successor plus the cost of the action that generated the successor. Formally,

$$h(s) \leq h(s') + c(a),\ a : s \rightarrow s',\ \forall (s, s') \in S,\ \forall a \in A.$$

Consistency implies admissibility, so a consistent heuristic is necessarily admissible, but an admissible heuristic need not be consistent. Inconsistency in a heuristic relates directly to the notion of inconsistency within the explicit graph discussed previously. When the underlying heuristic is inconsistent, the path to a state is not guaranteed to be optimal when that state is expanded the first time. In fact, when a heuristic is admissible—but inconsistent—A* may perform an exponential number of re-expansions, in the worst-case [21]. However, when combined with a consistent heuristic, A* is proven to find the optimal solution while expanding the fewest states of any algorithm using the same heuristic, up to tie-breaking.

UCS and A* are designed to find optimal solutions to the problems they solve, but in some cases it is not feasible to find an optimal solution, whether due to memory or time constraints. In the presence of these constraints, other techniques are necessary.

2.3 Bounded-Suboptimal Search

Bounded-suboptimal search (BSS) techniques are useful when finding an optimal solution is infeasible because of time or memory constraints. Additionally, there may be problems for which it is feasible to find an optimal solution, but it is not necessary for the solution to be optimal, only within some bound of optimal. BSS algorithms will find a suboptimal solution to a problem but ensure that the suboptimality of that solution is bounded by some value $\epsilon \geq 1$,

such that $c(\pi) \leq \epsilon \times C^*$. BSS allows for increased problem scalability compared to exact algorithms by expanding significantly fewer states, reducing memory overhead, and improving runtime. Furthermore, the parameter ϵ can be tuned to provide the desired trade-off between solution suboptimality and performance. When $\epsilon = 1$, the returned solution will be optimal. Generally, the less restrictive the bound, the better the performance.

2.4 Anytime Search

While bounded-suboptimal search algorithms perform well when given a desired suboptimality bound, there are occasions when, rather than a desired bound, there is an allotted amount of time within which a search algorithm must run. Once this time has expired, the algorithm must return the best solution found, along with a bound on the solution's suboptimality. Even more challenging, it might be that the search algorithm is allowed to run for an indeterminate amount of time, with the expectation that it can be terminated at *any* time and the best solution discovered should be returned. This gives rise to the class of *anytime* search algorithms, named for the requirement of termination at any time. Because anytime algorithms provide a bound on the suboptimality of returned solutions, they can also be used to find solutions that satisfy a desired bound like BSS techniques. Algorithm 2.2 gives general pseudocode for the framework of anytime best-first search.

Anytime algorithms have several key differences from best-first search and bounded-suboptimal search:

- The algorithm will terminate if the time bound t_{max} is exceeded.
- The algorithm does not necessarily terminate upon finding an initial solution.
- Once a solution is found, the cost of that solution is used to prune regions of the state space.
- The suboptimality bound of the current solution is returned when the algorithm terminates.

- When $\epsilon = 1$, an anytime algorithm will *converge* to an optimal solution if it is not terminated early. Unlike bounded-suboptimal algorithms, this does not mean that the algorithm's *initial* solution is guaranteed to be optimal.

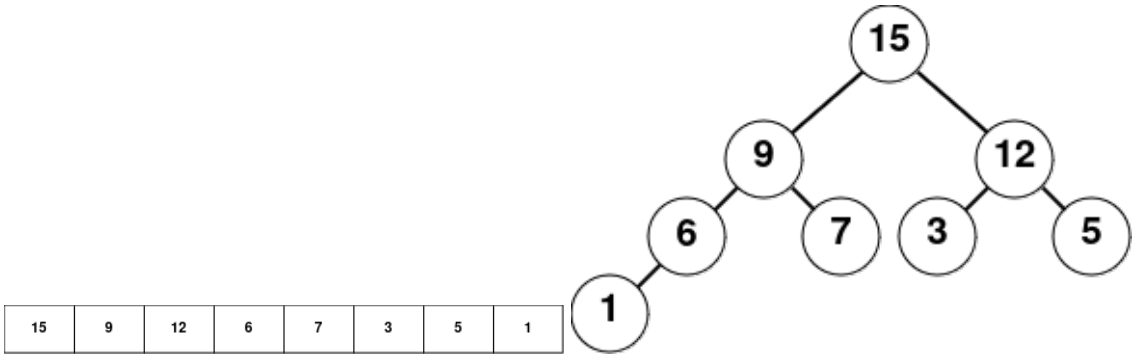
Like best-first search in general, the primary distinction between individual anytime algorithms is the state evaluation function. Another potential difference is the means by which anytime algorithms determine the suboptimality bound of the current solution.

2.5 Priority Queue Data Structures

A priority queue is an abstract data structure that maintains a mutable set of elements, each of which is associated with a priority (in general, these are real numbers). Elements are removed from the queue in priority order, and removal of all elements from the set constitutes sorting them by their priorities. The OPEN lists in algorithms 2.1 and 2.2, in practice, are implemented as priority queues. When used in these algorithms, the queues must support three core operations: *Enqueue*, which inserts a new element into the queue, storing it for later retrieval; *Dequeue*, which returns the next element in priority order, removing it from the queue; and *ChangeKey*, which updates the priority of an existing element. These three operations are sufficient for Dijkstra's algorithm, Uniform Cost Search. For some bounded-suboptimal and anytime search algorithms, it can be useful to have access to two additional operations: *Reorder* and *LowerBound*. The *Reorder* operation reconstructs the priority queue and is necessary when the priorities of all elements in the queue, assigned by the state evaluation function, may be altered simultaneously. This can occur when the evaluation function depends on a property that changes during the search, such as the suboptimality bound. The *LowerBound* operation returns the smallest f-value currently present in the priority queue, f_{min}. This enables computing the tightest possible bound on the suboptimality

Algorithm 2.2: General pseudocode for anytime best-first search.

```
Input: A, the set of actions
       s_0, the initial state
       S_T, the set of goal states
       c, the cost function
       ϵ, the desired suboptimality bound
       t_max, the allotted time limit
OPEN ← {s_0}
CLOSED ← ∅
ϵ̂ ← ∞, t̂ ← 0
incumbent ← ∅, G ← ∞
while OPEN ≠ ∅ and ϵ̂ > ϵ and t̂ < t_max do
    s ← min_{s∈OPEN} v(s)
    OPEN ← OPEN \ {s}
    if f(s) < G then
        CLOSED ← CLOSED ∪ {s}
        foreach (s′, c_a) ∈ {(a(s), c(a)) | a ∈ A} do
            if f(s′) < G then
                if s′ ∈ T then
                    incumbent ← s′
                    G ← g(s′)
                else
                    if s′ ∉ OPEN ∪ CLOSED then
                        g(s′) ← g(s) + c_a
                        OPEN ← OPEN ∪ {s′}
                    else if g(s′) > g(s) + c_a then
                        if s′ ∈ OPEN then
                            OPEN ← OPEN \ {s′}
                        else
                            CLOSED ← CLOSED \ {s′}
                        g(s′) ← g(s) + c_a
                        OPEN ← OPEN ∪ {s′}
            t̂ ← t̂ + Δt
            update ϵ̂
return (incumbent, ϵ̂)
```



(a) Array representation (b) Implicit tree

Figure 2.1: A binary heap of integers, where an integer's value is its priority.

of a solution, and so it is useful for anytime algorithms and some bounded-suboptimal algorithms, too.

As an abstract data structure, priority queues can be implemented in several ways, and the efficiency of a priority queue's operations can have a profound effect on runtime performance, making the choice of implementation a crucial decision. The most naive implementation that satisfies the necessary operations is a simple unordered array. Surprisingly, some operations can be performed efficiently; for example, Enqueue, ChangeKey, and Reorder are constant time. However, the Dequeue and LowerBound operations are linear time. In general, an array is a poor choice for a priority queue, and the more clever approaches discussed next are preferred [1].

2.6 Binary Heap

The most common priority queue in the literature is the *binary heap*, introduced as part of the heap sort algorithm [30]. A binary heap stores elements within an array $A[0...n-1]$, such that $key(A[i]) \geq key(A[j])$ $s.t.$ $1 \leq j \leq n-1$, $i = j \div 2$, where $key(A[i])$ and $key(A[j])$ are

[1] In the special case of Dijkstra's algorithm being performed on a *dense* graph, a better algorithmic complexity is achieved by using an array as the priority queue [5].

the priorities of the elements at index i and index j, respectively. This effectively maps the array to a *complete* binary tree—one where all levels are full except, possibly, the last, and leaf nodes are left aligned—with the added constraint that each node in the tree must have a priority that is greater-than-or-equal to the priorities of its children [2]. These constraints are known as the *heap property*.

By the heap property, any node k in the heap has the maximal priority of all nodes in the sub-tree rooted at k. Because the heap is mapped to a complete binary tree, the height of the heap is logarithmic in the number of elements it contains. The root of the heap—and therefore the highest priority element—is stored at the zeroth index of the array. For an index $i \geq n \div 2$, corresponding to a non-leaf node, the indices of its children (if any) can be computed simply. The index of the left child is $l_i = 2 \cdot i + 1$, and the index of the right child is $r_i = 2 \cdot i + 2$. Figure 2.1 shows both the array representation of a binary heap and its implicit tree for an arbitrary set of integers.

2.6.1 Operations and Complexity Analysis

By leveraging the structure of a complete binary tree, as well as the heap property, a binary heap can efficiently perform most priority queue operations on an arbitrary range of priority values. Additionally, by storing elements in an array, the memory overhead of the binary heap is minimal.

Enqueue. The first step of the Enqueue operation is to place the element at the next available index at the end of the array. This corresponds to this element becoming the rightmost leaf node in the implicit tree. The next step is to reassert the heap property. Because the heap property

[2]Note that in the case of A* (and similar search algorithms), a state with a *lower* f-value is considered to have *higher* priority.

was satisfied before adding the new element, it is the only element that may be in violation of the property. If the new element has a lower priority than its parent then the heap property is satisfied and the operation is complete. However, if the new element has a higher priority than its parent, the two elements are swapped. This process continues until the heap property is satisfied. Algorithms 2.3 and 2.4 provide complete pseudocode for the operation.

In the worst-case, the enqueued element will be swapped with its parent until it becomes the root. The number of swaps is then equal to the height of the tree, and the worst-case complexity of the Enqueue operation is $\Theta(\log n)$, where n is the number of elements in the heap.

Algorithm 2.3: Enqueue(A, s, pr)

Input: A, the array containing the heap of n elements
s, the state to enqueue
pr, an operator that returns the priority of a state

```
i ← n
n ← n + 1
A[i] ← s
HeapifyUp(A, i, pr)
```

Algorithm 2.4: HeapifyUp(A, i, pr)

Input: A, the array containing the heap
i, the index to begin from
pr, an operator that returns the priority of a state

```
while pr(A[i]) > pr(A[parent(i)]) and i ≠ 0 do
    swap(A[i], A[parent(i)])
    i ←parent(i)
```

Dequeue. By the heap property, the root of the binary heap is the highest priority element. Therefore, the Dequeue operation must remove and return the element stored at the root of the heap. The difficulty is in reasserting the heap property after removal of the root. This is accomplished

by first swapping the root with the rightmost leaf node, considering the former root to no longer be part of the heap. The new root is now the only element that may be in violation of the heap property. If the root has higher priority than either of its children, then the heap property is satisfied and the former root element is returned. Otherwise, the root is swapped with its highest priority child. The process will continue until the heap property is satisfied and then the original root is returned, as shown in Algorithms 2.5 and 2.6.

In the worst-case, the new root may be swapped downward until it becomes a leaf node once more. Again, the number of swaps is equal to the height of the tree, and, like the Enqueue operation, the worst-case complexity of the Dequeue operation is $\Theta(\log n)$.

Algorithm 2.5: Dequeue(A, pr)

```
Input: A, the array containing the heap of n elements
       pr, an operator that returns the priority of a state
n ← n − 1
swap(A[0], A[n])
HeapifyDown(A, 0, pr)
return A[n]
```

Algorithm 2.6: HeapifyDown(A, i, pr)

```
Input: A, the array containing the heap
       i, the index to start from
       pr, an operator that returns the priority of a state
if pr(A[i]) ≥ pr(A[leftChild(i)]) and pr(A[i]) ≥ pr(A[rightChild(i)]) then
    return
j ←leftChild(i)
if pr(A[j]) < pr(A[rightChild(i)]) then
    j ←rightChild(i)
swap(A[i], A[j])
HeapifyDown(A, j, pr)
```

ChangeKey. The ChangeKey operation has two informational requirements: first, the index of the element the operation is to be performed on, and second, the updated priority value of the element. Typically, this operation is used to increase the priority of an existing element, but in general, the priority may also be reduced. Given the index and the new priority, the operation consists of reasserting the heap property on the desired element. Assuming that the priority is increased, assertion of the heap property is performed in the same manner as in the Enqueue operation. Likewise, if the priority has been reduced, the process is the same reasserting the heap property in a Dequeue operation. Algorithm 2.7 shows pseudocode for the ChangeKey operation, assuming that the priority has been increased—modification to support the alternate case is trivial.

Because the ChangeKey operation is analogous to either the Enqueue or Dequeue operations, depending on the circumstance, the complexity of the operation is also analogous. Therefore, the worst-case complexity of ChangeKey is $\Theta(\log n)$.

Algorithm 2.7: ChangeKey(A, i, pr)

Input: A, the array containing the heap
i, the index to update
pr, an operator that returns the (up-to-date) priority of a state

```
1 HeapifyUp(A, i, pr)
```

LowerBound. The LowerBound operation is relevant when a priority queue is used in conjunction with a search algorithm that orders state expansions by a priority other than f-value. Because LowerBound should return f_{min}, the structure of the binary heap cannot be leveraged to accelerate this operation. Given an operator that returns the f-value of an element, the LowerBound operation

is performed by linearly scanning the array representing the heap to find f_{min} and return it, as shown in Algorithm 2.8. The worst-case complexity of this operation is $\Theta(n)$.

Algorithm 2.8: LowerBound(A, f)

```
Input: A, the array containing the heap of n elements
       f, an operator that returns the f-value of a state
minIndex ← 0, i ← 1
while i < n do
    if f(A[i]) < f(A[minIndex]) then
        minIndex ← i
    i ← i + 1
return f(A[minIndex])
```

Reorder. The Reorder operation is relevant when the priorities of all (or most of) the elements in the queue change simultaneously. Given an operator that returns the up-to-date priorities of the elements, a naïve approach would be to construct a new binary heap by calling Enqueue on all of the elements, using their new priority values. Although this method is correct in that it produces a proper binary heap, it is slow. The complexity of building a heap in this way is $\Theta(n \log n)$ in the worst case. Fortunately, this complexity can be reduced by making use of Floyd's algorithm, shown in Algorithm 2.9. Floyd's algorithm constructs a binary heap from an arbitrary array of elements in $\Theta(n)$ time in the worst case [5]. Thus, using Floyd's algorithm, the complexity of the Reorder operation is also $\Theta(n)$ in the worst case.

Memory. Because the required size of the binary heap is not known a priori, a doubling strategy is used to grow the array when necessary—this strategy of geometric growth amortizes the costs of resizing to constant time. Let n be the maximal number of simultaneously enqueued elements.

Algorithm 2.9: Reorder(A, pr)

Input: A, the array containing the heap of n elements
pr, an operator that returns the priority of a state

```
1 i ← ⌊n ÷ 2⌋
2 while i ≥ 0 do
3     HeapifyDown(A, i, pr)
4     i ← i − 1
```

Because the array grows by doubling, the size of the array is at most $2n-1$. The memory complexity of the binary heap is then $\Theta(n)$.

The binary heap provides reasonably good performance for the core priority queue operations, but suffers with respect to the LowerBound and Reorder procedures. Despite this, the binary heap is an excellent general purpose priority queue. It can operate effectively on any range of priority values, whether they are discrete or continuous values. But there are more specialized structures that make use of specific domain characteristics to achieve superior performance under certain conditions.

2.7 Bucket Queue

When restrictions exist on the domain of priority values, it is possible to exploit them to create more efficient priority queues. A common technique is *bucketing*; that is, mapping elements that have the same, or similar, priority values into the same discrete location, known as a bucket. Because priority values are mapped directly to buckets, the cost of insertions and look-ups is reduced substantially. Although there has been work on *quantized* bucket queues—bucket queues that operate on real-valued priorities, quantizing (or discretizing) them into their buckets—the bucketing approach is most effective when priorities are non-negative integers, spanning a relatively

small range [24]. The bucket-based priority queues discussed in this dissertation are designed for the second scenario.

The first bucket queue for best-first search, sometimes called a *Dial Queue* after its inventor, was originally introduced to improve Dijkstra's algorithm, though it is also applicable to A* [6][2]. To simplify the discussion, the focus will be on Dijkstra's algorithm. At its simplest, a bucket queue is an array of buckets $A[0...C^*]$ that maps the g-values of states directly to indices (buckets). Each bucket is a doubly-linked list that contains states with the same priority. A reference is maintained to the smallest non-empty bucket, denoted g_{min}.

There are two properties of the underlying search graph that are useful for optimizing the bucket queue: the maximum transition cost and the monotonicity of priorities. The maximum transition cost, T, limits the maximum change in priority between a successor and its parent. Because expansions are performed in order of g-value, T defines the maximum range of priorities that can be in the bucket queue. Leveraging this, instead of using an array $A[0...C^*]$, the bucket queue can be implemented as an array $A[0...T]$. Priorities then map to buckets via modulo division by the number of buckets. Typically, the maximum transition cost is significantly smaller than C^*. In unit-cost domains, $T = 1$.

The priority values in Dijkstra's algorithm are monotone; that is, they are non-decreasing. This means that the g-value of a successor is always at least that of its parent. This fact ensures that having $T + 1$ buckets is always enough. Furthermore, this reduces the difficulty of maintaining the reference to g_{min}, an additional operation that the bucket queue must support. Figure 2.2 provides an example of a bucket queue on some arbitrary values.

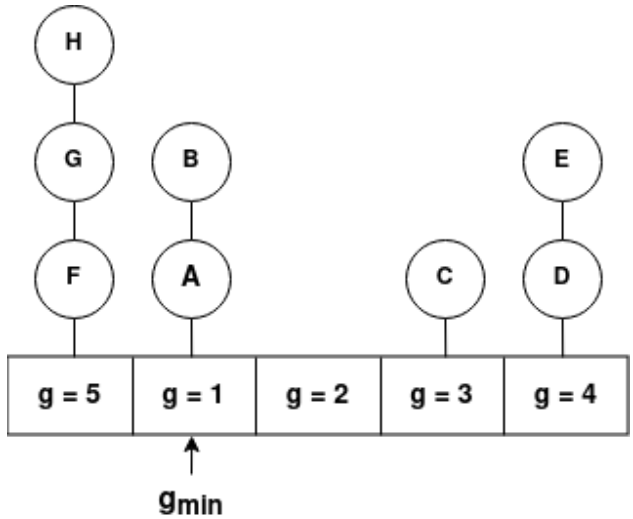


Figure 2.2: A bucket queue of arbitrary states, where $T = 4$.

2.7.1 Operations and Complexity Analysis

By exploiting constraints on the priority domain and properties of the underlying search graph, the bucket queue can perform priority queue operations with great efficiency. However, doing so requires an additional priority queue operation, *UpdateMin*, which ensures that the g_{min} reference is up-to-date. Because the bucket queue is intended for use with Dijkstra's algorithm (or A*), the Reorder and LowerBound operations are not applicable to the structure.

UpdateMin. When a Dequeue operation is performed, it may be necessary to update the g_{min} reference. The complexity of this update requires careful consideration, so it will be discussed separately from Dequeue. When the linked list in the g_{min} bucket becomes empty, the next smallest non-empty bucket must be identified and g_{min} must be updated accordingly. Because the g-values are monotone, g_{min} cannot decrease. Therefore, a linear scan is performed, starting from the current g_{min} index and terminating when a non-empty bucket is found. In the worst-case, this procedure

must scan all of the buckets, and so has complexity $\Theta(T)$. However, it is more useful to analyze this operation using an aggregate analysis.

Aggregate analysis, a form of amortized analysis, considers how the cost of individual operations relate to the entire sequence of operations applied to a data structure. An aggregate analysis can better characterize the true cost of operations, providing a tighter bound than a simple worst-case analysis [26]. The key to analyzing UpdateMin is that the g-values are monotone. The total range of g-values encountered during the search is $[0, C^* + T]$, and the maximum value reached by g_{min} is C^*. So, g_{min} will traverse the range $[0, C^*]$ exactly once. The aggregated cost of all UpdateMin operations throughout the search is then $\Theta(C^*)$. When the number of states generated during the search is proportional to C^* (or greater), the cost of UpdateMin amortizes to constant[3]. Algorithm 2.10 shows the process of updating g_{min}, assuming that the queue is not empty.

Algorithm 2.10: UpdateMin(A, i)

```
Input: A, the array of buckets
       i, the previous minimum index
1 j ← (i − 1 + |A|) mod |A|
2 while A[i] = ∅ and i ≠ j do
3     i ← (i + 1) mod |A|
4 min_A ← i
```

Enqueue. The first step of the Enqueue operation is to identify the target bucket. The index corresponding to the bucket is computed by the state's g-value, modulo the number of buckets. After determining the appropriate bucket, the state is added to the linked list stored in the bucket.

[3]In unit-cost domains this is guaranteed to be the case, but when the transition costs are non-uniform it is not guaranteed.

Because all states in this bucket have the same priority, it does not matter where the state is placed within the list. Algorithm 2.11 shows the simple process. The costs of computing the bucket index, accessing the array, and appending to the linked list are all constant. Therefore, the complexity of the bucket queue's Enqueue operation is $\Theta(1)$.

Algorithm 2.11: Enqueue(A, s, pr)

Input: A, the array of buckets
s, the state to enqueue
pr, an operator that returns the priority of a state

1 $i \leftarrow pr(s) \mod |A|$
2 $A[i] \leftarrow A[i] \cup s$

Dequeue. Assuming that g_{min} is up-to-date and the queue is not empty, the Dequeue operation is simple to perform. The bucket referenced by g_{min} is accessed and an arbitrary state is removed from the linked list to be returned. If removing this state empties the bucket, then the UpdateMin procedure must be performed, as seen in Algorithm 2.12. The cost of UpdateMin has been analyzed separately, so it will not affect the analysis of Dequeue. Accessing the g_{min} bucket and removing a state from the linked list are both constant time operations, so the complexity of the Dequeue operation is $\Theta(1)$.

Algorithm 2.12: Dequeue(A)

Input: A, the array of buckets

1 $s \leftarrow x \in A[min_A]$
2 $A[min_A] \leftarrow A[min_A] \setminus s$
3 UpdateMin(A, min_A)
4 **return** s

ChangeKey. The ChangeKey operation requires two pieces of information: a pointer, or reference, to the existing state in the queue and the updated priority of the state. The reference to the existing state is used to remove it from its current linked list, and the new priority is used to compute the new bucket index. The state is then added to the linked list in the new bucket (Algorithm 2.13). Effectively, the ChangeKey operation is the combination of a Dequeue operation (albeit from an arbitrary location) and an Enqueue operation. Both of these operations are constant time, so the complexity of ChangeKey is $\Theta(1)$.

Algorithm 2.13: ChangeKey$(A, s, pr, \hat{pr})$

Input: A, the array of buckets
s, a reference to the existing state in the queue
pr, an operator that returns the priority of a state
$\hat{pr}$, the old priority of s

```
1 i ← p̂r mod |A|
2 A[i] ← A[i] \ s
3 if i = min_A and A[i] = ∅ then
4 |   UpdateMin(A, i)
5 Enqueue(A, s, pr)
```

Memory. Let n be the maximal number of states in the bucket queue simultaneously. The total memory overhead of the bucket queue is then $\Theta(n + T)$. When n is proportional to T (or greater), the memory complexity simplifies to $\Theta(n)$.

When a domain satisfies the necessary criteria for using a bucket queue—discrete priorities and a relatively small value of T—it can improve the performance of Dijkstra's algorithm (or A*), compared to a binary heap. In fact, under the appropriate conditions, the bucket queue improves the asymptotic complexity of Dijkstra's algorithm [6].

CHAPTER 3

TWO-LEVEL BUCKET QUEUE FOR A*: ANALYSIS AND IMPROVEMENTS

This chapter begins by describing the a two-level bucket queue data structure for the priority queue for A*. A detailed complexity analysis is provided—an analysis that is absent from the literature. Next, a simple modification of the two-level bucket queue is established, which improves the structure's worst-case complexity; this modification is foundational to the novel priority queue introduced in Chapter 4. Most importantly, a novel bucket-based priority queue for A* is presented: the lazy two-level bucket queue. This priority queue improves the asymptotic complexity of the A* algorithm with tie-breaking compared to the existing two-level bucket queue. Lastly, a simple method to improve hash table performance in heuristic search is discussed, inspired by the way bucket-based priority queues exploit search order.

3.1 Two-Level Bucket Queue for A*

Although it is possible to use a traditional bucket queue with A*, there is one major drawback to doing so: tie-breaking. In the bucket queue, expansion order is not structured between states that have equivalent priorities (f-values). It is common practice for the A* algorithm to tie-break equivalent f-value states by expanding them in non-decreasing order of their h-values; a strategy that cannot be performed efficiently when using a traditional single-level bucket queue.

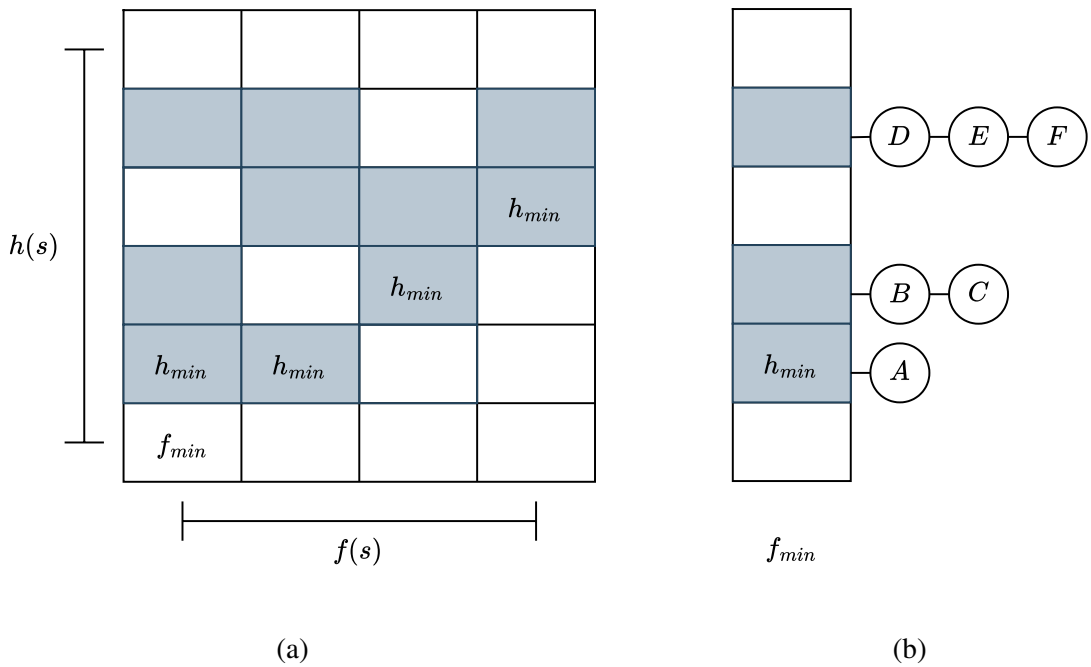


Figure 3.1: (a) A high-level view of an arbitrary two-level bucket queue, omitting the third dimension. Shaded cells indicate non-empty secondary buckets. (b) An expanded view of the f_{min} primary bucket. The nodes A–F represent arbitrary states.

The *two-level bucket queue* (or two-level queue) is a two-dimensional variant of the bucket queue that efficiently enables the A* tie-breaking strategy. This data structure is often used in implementations of A*, although the only previous description of it in the literature appears to be in [2]. The queue's first level is, in effect, the same as Dial's bucket queue and is indexed by f-value. Each bucket in the first level—known hereafter as a primary bucket—contains another bucket queue. These second-level queues are indexed by h-value (or g-value) and their buckets, which will be referred to as secondary buckets, contain doubly linked lists of states. States residing in the same secondary bucket have identical f-, g-, and h-values. Figure 3.1 shows a representation of a two-level bucket queue.

Like the bucket queue, the two-level queue must maintain a reference to the first non-empty primary bucket, denoted f_{min}. Again, mirroring the bucket queue, the first level of the queue can be implemented as an array $A[0...T]$; as before, priorities map to primary buckets through modulo division[1]. Unlike the bucket queue, the two-level queue must also maintain references to the first non-empty secondary bucket in each of the second-level queues. These will be referred to as h_{min} references.

3.1.1 Operations and Complexity Analysis

A detailed complexity analysis of the two-level queue appears to be absent from the literature, and existing claims about its complexity ([2]) are unfortunately incomplete. To rectify this situation, a detailed analysis of each operation is provided here. As was the case for the bucket queue, the Reorder and LowerBound operations are not applicable.

UpdateMin. The most important operation to consider is UpdateMin because it underpins the complexity of the entire structure. It is also the most difficult to analyze. For clarity, the analysis is considered in two parts: first, the maintenance of the f_{min} reference, and second, the maintenance of the h_{min} references. The complexity of maintaining the f_{min} reference depends on the search heuristic satisfying the consistency property; in other words, the heuristic must be monotone. When this is the case, the f-values of states are monotone and the complexity of maintaining f_{min} is analogous to that of maintaining g_{min} in the bucket queue. The aggregated cost of maintaining

[1]Note that while T still represents the maximum transition cost in the underlying graph, its meaning is slightly altered in the context of A*, representing the maximum change in f-value, rather than just g-value.

f_{min} throughout the search is then $O(C^*)$, and when the number of states generated during the search is proportional to C^* (or greater), the cost is amortized to constant[2].

When a secondary bucket corresponding to an h_{min} reference becomes empty, the next non-empty secondary bucket must be identified, and h_{min} must be updated. Similar to maintaining f_{min}, this is achieved by linearly scanning the secondary buckets until a non-empty bucket is identified[3]. Unfortunately, the complexity of this update cannot be aggregated as it can for f_{min} because the h-values are not monotone.

Lemma 1. *The worst-case complexity of updating an h_{min} reference is $O(C^*)$.*

Proof. The range of h-values—and therefore secondary buckets—depends upon the quality of the heuristic estimate. Under the assumption of admissibility, the heuristic will never overestimate the cost of reaching the goal. Because A* will never expand a state that exceeds the optimal solution cost, it follows that the maximum cost of updating an h_{min} reference will never exceed the optimal solution cost C^*. □

Example. It is not immediately clear how this worst-case complexity could be encountered in practice. Consider a scenario in which the f_{min} bucket has an h_{min} value equal to C^* and the h_{min} bucket contains two or more states. Suppose a series of $C^* - 1$ expansions occurs where for each expansion, a single successor is inserted into the f_{min} primary bucket, becoming the new h_{min} value. If the next expansion yields no successor with an f-value equal to f_{min}, then h_{min} will have

[2] In unit-cost domains the length of a solution and its cost are equivalent, making this requirement trivial. However, when costs are not uniform, the cost of an optimal solution may greatly exceed its length. In this case, it is possible (though unlikely) that the cost of maintaining f_{min} does not amortize to constant.

[3] Because these update procedures are nearly identical to the bucket queue's, pseudocode for them has been omitted.

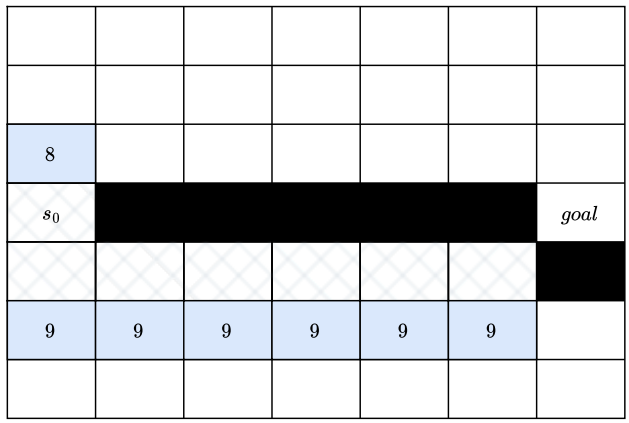


Figure 3.2: An example where the worst-case complexity of UpdateMin occurs in the two-level queue. S_0 indicates the initial state, fully-shaded cells mark impassable regions, cross-hatched cells mark expanded states, and the states on the OPEN list are shaded, with their f-values labeled.

to be updated at a cost of $C^* - 1$. Figure 3.2 shows a trivial situation where this occurs[4]. So, by Lemma 1 the worst-case complexity of UpdateMin is $O(C^*)$.

Enqueue. The Enqueue operation of the two-level queue naturally extends that of the bucket queue. The state's f-value is used to identify the appropriate primary bucket, its h-value is used to locate the necessary secondary bucket, and the state is inserted into the linked list at this location. If the state's h-value is less than the primary bucket's current h_{min} value, then h_{min} is updated to the new value in constant time. See Algorithm 3.1 for pseudocode of the procedure.

Each of the steps performed are constant time; therefore, the complexity of the Enqueue operation is $\Theta(1)$.

[4]One might consider amortizing the cost of the h_{min} update in this example. Indeed, in this trivial case it is possible to do so. However, this would not account for the effect that the expansions have on the internal structure of subsequent primary buckets. Furthermore, an amortized cost to maintain h_{min} within a single primary bucket must account for the maximum possible change in h-value in domains that are not unit-cost, complicating the analysis.

Algorithm 3.1: Enqueue

Input:
A, the primary bucket queue
s, the state to enqueue
f, an operator that returns the f-value of a state
h, an operator that returns the h-value of a state

```
1 B ← A[f(s) mod |A|]
2 B[h(s)] ← s
3 if h(s) < h_min then
4 |   h_min ← h(s)
```

Dequeue. The Dequeue operation, shown in Algorithm 3.2, is where the tie-breaking provided by the two-level queue is utilized. Assuming that f_{min} is up-to-date, the primary bucket it references is accessed. Again, assuming that it is up-to-date, the secondary bucket referenced by h_{min} is accessed, and an arbitrary state is removed from its linked list, to be returned. If removing the state empties the secondary bucket, then h_{min} must be updated. Furthermore, if removing the state leaves all secondary buckets empty, then f_{min} must be updated.

The cost of updating f_{min} has been aggregated across the search, and, as before, it will not be considered a part of Dequeue's cost. However, the cost of updating h_{min} could not be aggregated, so its cost does affect the Dequeue operation. Apart from the h_{min} update, the steps of the Dequeue operation are constant time. The complexity of Dequeue then depends on that of updating h_{min}. As such, the worst-case complexity of the Dequeue operation is $O(C^*)$.

ChangeKey. Much like for the bucket queue, the ChangeKey operation requires a pointer or reference to the existing state in the queue, and its updated priority. The operation is then comprised of a Dequeue from the state's current location and an Enqueue into its new location. The reference to the existing state is used to remove it from its current location, and the updated

Algorithm 3.2: Dequeue

```
Input:
A, the primary bucket queue containing r buckets
f, an operator that returns the f-value of a state
h, an operator that returns the h-value of a state
B ← A[f_min]
s ← x ∈ B[h_min]
B[h_min] ← B[h_min] \ s
// Conditionally update h_min
if B[h_min] = ∅ then
    UpdateMin(B, f, h)
// Conditionally update f_min
if A[f_min] = ∅ then
    UpdateMin(A, f, h)
return s
```

priority is used to identify the new primary bucket for insertion [5]. The state is then added to the linked list at the new location. Algorithm 3.3 shows the pseudocode for the operation.

Because the operation is composed of Dequeue and an Enqueue, care must be taken to ensure that the f_{min} and h_{min} references are maintained. So, the complexity of the ChangeKey operation depends on these update costs, making it $O(C^*)$ in the worst case.

Memory Overhead. Analyzing the memory overhead of the two-level queue is simpler than analyzing its operational complexity. First, like the bucket queue, the memory overhead of the primary buckets depends on the maximum transition cost. The maximum range of secondary buckets is $O(C^* + T)$. Therefore, the memory overhead of the two-level bucket structure is $O(T \cdot C^* + T^2)$, and the total memory complexity of the two-level queue is $O(n + T \cdot C^* + T^2)$, where n is the maximum number of states in the queue simultaneously. When n is the dominant term, the overhead of the bucket structure drops from the analysis.

[5] The h-value of the state will not differ from before, so only the updated f-value is required.

Algorithm 3.3: ChangeKey

Input:
A, the primary bucket queue containing r buckets
s, a reference to the existing state in the queue
f, an operator that returns the f-value of a state
h, an operator that returns the h-value of a state
$\hat{f}$, the old f-value of s

```
1 B ← A[f̂ mod |A|]
2 B[h(s)] ← B[h(s)] \ s
3 if B[h(s)] = ∅ and h(s) = h_min then
4 |   UpdateMin(B, f, h)
5 if A[f̂ mod |A|] = ∅ then
6 |   UpdateMin(A, f, h)
7 Enqueue(A, s, f, h)
```

In practice, the worst-case complexity of the Dequeue and ChangeKey operations is rarely encountered, and the two-level bucket queue can provide substantial performance improvements over a binary heap for A*. However, it is possible to reduce the worst-case complexity exponentially with a simple modification.

3.2 Modified Two-Level Bucket Queue with Improved Worst-Case Complexity

To motivate the need for an improved worst-case complexity, consider again the trivial example in Figure 3.2. In this scenario, the cost of the last Dequeue operation on the two-level queue is exponentially more than the cost of the same operation on a binary heap. So, under the appropriate conditions, it is possible for the two-level queue to be asymptotically worse than a binary heap!

The solution to mitigating this worst-case behavior is simple: use binary heaps! For each primary bucket of the two-level queue, maintain a binary heap of (indices to) the non-empty secondary buckets. These binary heaps will be referred to as *active heaps* to emphasize that operations on them depend only on the active (non-empty) secondary buckets.

3.2.1 Complexity Analysis

The simple modification described above impacts the complexity of every operation on the two-level queue, reducing the worst-case complexity exponentially.

UpdateMin. As before, the cost of maintaining f_{min} is aggregated across the search. However, where before the complexity of updating an h_{min} reference was linear in the range of secondary buckets, it is now logarithmic in the size of the corresponding active heap. In the worst case, every secondary bucket can be non-empty, resulting in a complexity of $O(\log C^*)$, per Lemma 1—a substantial improvement!

Enqueue. Before, the cost of maintaining h_{min} references did not impact the Enqueue operation because the cost of reducing h_{min} was constant. Now, when the value of h_{min} is reduced, a new index must be inserted into the corresponding active heap. More generally, if an Enqueue operation makes *any* previously empty secondary bucket non-empty, the index of that bucket must be inserted into the appropriate active heap. In either case, the cost is logarithmic in the size of the active heap, and the worst-case complexity of the Enqueue operation is $O(\log C^*)$.

Dequeue and ChangeKey. Like with the original two-level queue, the complexity of the Dequeue and ChangeKey operations depends on that of the UpdateMin operation. As such, the worst-case complexity of both the Dequeue and ChangeKey operations is $O(\log C^*)$, another significant reduction in complexity.

Although these improvements in worst-case complexity are impressive, it does not necessarily mean that there will always be a performance difference between these two structures in practice;

or, indeed, that the asymptotically improved version cannot be slower in practice. Table 3.1 compares the best-case and worst-case complexities of the binary heap, two-level queue, and modified two-level queue, to aid in a more detailed comparison of these structures [6].

Table 3.1: The best-case and worst-case complexities of operations for each of the priority queues.

	Best Case			Worst Case		
	Enqueue	Dequeue	ChangeKey	Enqueue	Dequeue	ChangeKey
Binary Heap	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$
Two-Level Queue	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$O(C^*)$	$O(C^*)$
Two-Level Queue (M)	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$O(\log C^*)$	$O(\log C^*)$	$O(\log C^*)$

In the best case, each of these structures has constant complexity for every operation. It may seem that this does not provide any useful insight, but the key is to consider when the best case occurs. In particular, for the Dequeue and ChangeKey operations of the two-level queue and its modified variant, the scenarios in which the best-case complexities occur are inverses. For a given primary bucket, if the number of non-empty secondary buckets is small, then it is more likely to encounter the worst-case complexity in the two-level queue, as in Figure 3.2[7]. However, this results in the best-case complexity for the modified two-level queue. Conversely, the best case for the two-level queue occurs when all secondary buckets are non-empty. This is the worst case for the modified two-level queue.

[6]Operations given in terms of C^* do not use Θ-notation because they actually depend on the underlying heuristic estimate, which may not actually relate to the optimal solution cost; consider a trivial heuristic that returns a value of 1 for every state except the goal state.

[7]This does depend on the distribution of the non-empty secondary buckets across the range, but considering that in detail requires an average case analysis that depends on properties of the state-space graph being searched.

Given this inverse relationship in the performance of the two-level queue and its modified variant, it is clear that the properties of the state-space graph impact the decision of which structure should be used in practice. The decision becomes even more nuanced when comparing the worst-case complexities of the two structures. In domains with large branching-factors it is possible for the number of Enqueues to exceed the number of Dequeues substantially enough that the cost of Enqueue operations is the most important consideration. In this case, the two-level queue would be more beneficial than the modified version. Alternatively, if the number of Dequeue or ChangeKey operations is the dominating factor, then it is more likely that the modified two-level queue will perform better; though, again, this depends on the characteristics of the best-case complexities mentioned previously.

Although the worst-case complexity of operations on the traditional two-level bucket queue can be worse than for a binary heap, the worst-case complexity of operations on the modified two-level bucket queue can never be worse.

Lemma 2. *Let x be the size of the largest active-heap, and let n be the number of states in the priority queue. Then* $\log n \in \Omega(\log x)$.

Proof. Given that an active heap only operates on non-empty secondary buckets, there are *at least* x non-empty secondary buckets in the modified two-level queue. By definition, a secondary bucket is non-empty iff it contains one or more states. Thus, at worst, $x = n$. □

So, by Lemma 2, the modified two-level queue can never be asymptotically worse than a binary heap. This worst-case guarantee makes the modified two-level queue useful as a foundation for extending bucket-based priority queues to algorithms other than A*, as will be done in Chapter 4.

3.3 Lazy Two-Level Bucket Queue for A*

Although they offer the advantage of tie-breaking for A*, neither the two-level queue nor its modified variant achieve the amortized constant complexity of Dial's original bucket queue—a property that would be nice to have! This section presents a new bucket-based priority queue, built upon the two-level bucket queue, that enables both tie-breaking and amortized-constant priority queue operations for the A* algorithm. This new structure is called the *lazy two-level bucket queue* (or lazy two-level queue).

The lazy two-level queue retains the same primary and secondary bucket structures as the traditional two-level bucket queue, indexed by f-value and h-value, respectively. As with the modified two-level queue, the main consideration is the cost of maintaining the h_{min} references. Rather than using active heaps, each primary bucket in the lazy bucket queue is associated with an *active list*, a doubly linked list of non-empty secondary buckets. It is easy to see that when an h_{min} secondary bucket becomes empty, the process of updating h_{min} is to simply remove the head of the corresponding active list in constant time. However, when a previously empty secondary bucket becomes non-empty, it must be inserted into the appropriate location within the active list. The worst-case complexity of this operation is $O(C^*)$. In effect, the cost associated with maintaining the h_{min} references has simply been moved from Dequeue to Enqueue.

The final step necessary to achieve the desired complexity comes from reviewing Dial's original insight: f_{min} is monotonic. Because A* only Dequeues from the f_{min} primary bucket, it is the only primary bucket for which information about h_{min} is needed. Therefore, this is the only primary bucket that needs an active list, and Dequeue operations are performed in constant time. If the successor of an expanded state is Enqueued into the f_{min} primary bucket, it must have an h-value

smaller than h_{min}, and it becomes the head of the active list—a constant time operation. And Enqueue operations into all other primary buckets are constant time, requiring no work to maintain an active list. The last consideration is when f_{min} increases: at this stage, the active list for the corresponding primary bucket is populated. This delayed, or *lazy* approach to constructing the active lists is where the lazy bucket queue gets its name, and it is vital to achieving the desired complexity result.

Theorem 1. *When used for A*, the operation costs of the lazy two-level bucket queue can be amortized to constant time.*

Proof. Let g and e be the number of states generated and expanded during A* search, respectively. Let E and D represent the worst-case complexity of the Enqueue and Dequeue operations for a given priority queue, respectively, and let W represent the cost of any additional priority queue overhead. The worst-case complexity of A* then takes the form $O(g \cdot E + e \cdot D + W)$.

Lemma 3. *The aggregated cost of lazily constructing the active lists is* $O(C^{*2})$.

Proof. Let x be the number of active lists constructed during the search. Because f_{min} increases monotonically and an active list is only constructed when f_{min} increases, it follows that $x \in O(C^*)$. To construct an active list, a linear scan is performed across the range of secondary buckets. Per Lemma 1, the cost of this linear scan is $O(C^*)$. Thus, the aggregated cost of constructing the active lists is $O(C^{*2})$. ■

The worst-case complexity of A* with tie-breaking and using the lazy two-level queue is then $O(g+e+C^{*2})$. When $g+e$ is proportional to C^{*2}, all overhead of the lazy bucket queue is amortized to constant. □

In contrast, the worst-case complexity of A* with tie-breaking is $O((g+e)\log C^* + C^*)$ when using the modified two-level queue and the priority queue overhead cannot be amortized away. The complexity result of Theorem 1 relates nicely to the complexity of Dial's algorithm, with the difference being the C^{*2} term, a natural consequence of the structure's two-dimensionality. Despite the improvement in asymptotic complexity, it is not necessarily the case that the lazy bucket queue will always outperform the two-level queue or its modified variant in practice. A focus on worst-case analysis is useful when an algorithm or data structure must satisfy certain guarantees, but it is often the case that for specific domains the worst case is known not to occur, and so other characteristics will influence which priority queue is best for a given domain.

3.3.1 Experimental Comparison

To emphasize that worst-case complexity is not the only consideration for priority queue selection, a simple experimental comparison is performed between the two-level bucket queue, the modified two-level bucket queue, and the lazy two-level bucket queue. Additionally, to show the effect of implementation decisions on the performance of these structures, two versions of the two-level bucket queue will be compared: one in which the secondary buckets are implemented as doubly linked lists and one in which the secondary buckets are implemented as dynamic arrays. The experimental domain used is Korf's 100 instances of the 15-puzzle [18], and the data presented is averaged across these instances.

15-puzzle. The 15-puzzle consists of a four-by-four grid that contains 15 numbered tiles and a blank space. The objective is to rearrange the tiles from their initial configuration into a predefined goal configuration; Figure 3.3 shows an example starting state and the desired goal state. An action

14	13	15	7
11	12	9	5
6		2	1
4	8	10	3

(a)

	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

(b)

Figure 3.3: (a) The initial configuration for instance 1 of Korf's 100 puzzles. (b) The desired goal configuration.

consists of sliding a tile into the blank space, where the moved tile must be cardinally adjacent to the blank space. Actions are unit-cost, and the Manhattan distance heuristic was used for all 15-puzzle experiments.

Dynamic Array vs. Linked List. Table 3.2 shows the speedup factor per operation achieved by using a dynamic array as the implementation method for secondary buckets instead of a doubly linked list. Clearly, while the doubly linked list is useful for explaining the structure and analyzing its complexity, an implementation that uses it will have its practical performance hindered. The intuition for this is that a doubly linked list will not store a state (or pointer/reference to a state) directly, instead wrapping the state in a list-node structure that maintains pointers to the previous and next items in the list. This requires a dynamic memory allocation for every insertion into the priority queue, increasing the overhead of insertion operations[8]. The two additional pointers per priority queue entry also increase the practical memory consumption; in fact, this increased

[8]By using a more complex memory allocation strategy, it is possible to avoid dynamically allocating memory for every insertion, but at the cost of increasing implementation difficulty. The improved allocation strategy would likely suffer from memory fragmentation issues over time.

Table 3.2: The speedup factor of using a dynamic array vs. a linked list as the secondary buckets in the two-level bucket queue.

	Operation		
	Enqueue	Dequeue	ChangeKey
Speedup	2.04x	1.05x	2.03x

memory requirement caused this version of the two-level bucket queue to be unable to solve instances 97, 99, and 100 from the test set[9].

By implementing the secondary buckets as dynamic arrays, the number of memory allocations required is reduced. When one of these dynamic arrays becomes full, a new array of twice the size (or some other geometric growth factor) can be allocated, and the entries from the old array can be moved into the new array. This strategy amortizes the cost of moving the data when resizing, retaining constant complexity for inserting into a secondary bucket. Another benefit of this approach is that secondary bucket data is contiguous in memory, allowing for more cache-friendly data accesses. The speedups shown in Table 3.2 indicate that the Enqueue and ChangeKey operations are significantly improved. This aligns with expected behavior, as those operations would require the linked lists to allocate memory, unlike the Dequeue operation. Given these results, all future experiments will implement the secondary buckets as dynamic arrays.

Comparing Two-Level Bucket Queue Variants. Table 3.3 shows the speedup factor per operation and two-level bucket queue variant, relative to the two-level bucket queue itself. Although the term speedup is used, the modified two-level queue and the lazy two-level queue are *slower* in practice than the two-level bucket queue (for this particular domain). This may be surprising at

[9]For consistency, all 15-puzzle experiments in this chapter will omit these instances.

Table 3.3: The speedup factor of each two-level bucket queue variant relative to the standard two-level bucket queue.

	Modified Two-Level Queue			Lazy Two-Level Queue		
Operation	Enqueue	Dequeue	ChangeKey	Enqueue	Dequeue	ChangeKey
Speedup	0.82x	0.92x	0.86x	0.74x	1.02x	0.83x

first, given that both variants have better worst-case complexity than the two-level bucket queue. However, the worst case rarely occurs in practice, and both the modified two-level queue and the lazy two-level queue have additional constant-factor overhead in the best and average cases when compared to the two-level bucket queue. For example, maintaining the f_{min} primary bucket's active list in the lazy two-level queue requires dynamic memory allocation; as shown in Table 3.2, memory allocation can have a significant impact on performance.

This further emphasizes that domain-specific criteria should be used when selecting an appropriate priority queue, not just worst-case operation complexities. For many domains, the standard two-level bucket queue may have the best performance, while for others it may be the modified two-level queue or the lazy two-level queue.

3.4 Exploiting Locality of Reference

In graph-search algorithms, the hash table that manages information about which states reside in the OPEN and CLOSED lists is accessed at least as often as the priority queue but is typically accessed *more* frequently. As such, the hash table implementation can have a significant impact on an algorithm's performance. When using an effective hash function, hash tables distribute the elements they contain (roughly) uniformly throughout the structure, minimizing collisions, reducing probe sequence length, and achieving constant-time operations on average. This makes

them effective general-purpose data structures for element storage and retrieval; however, they do not leverage any structural relationships that may exist between the elements they store.

The bucket-based priority queues discussed so far have leveraged properties of both the state space and the A* algorithm itself to achieve improved asymptotic complexity during search. It is also possible to exploit some of these properties to improve hash table performance, albeit practically, not asymptotically. The uniform distribution of elements within a hash table enables efficient look-ups, independent of the number of elements; although an excellent theoretical characteristic, the propensity for each look-up to require accessing significantly different memory addresses is not cache-friendly from a hardware perspective. As the size of the hash table grows, the likelihood of cache misses increases, degrading performance.

Consider the expansion of a single state during A* search. This parent state and its successors are spatially local to each other within the state space, but when performing duplicate detection on the successors, it is possible for each operation to incur a cache miss (or more than one). This is because, despite their spatial locality within the state space, they will not have spatial locality within the hash table. Ideally, no more than a single cache miss would be incurred by duplicate detection on the successor set. Unfortunately, the modification discussed next does not achieve this lofty goal; however, this change, simple though it is, proves to be effective at reducing the average operation time for hash tables in A* search.

Instead of a single monolithic hash table, an array of hash tables $A[0...C^*]$ is used. This array is indexed by the heuristic estimate of the state being stored or retrieved. Therefore, the hash table at index i is responsible only for states that are estimated to be distance i from the goal. The heuristic estimate provides approximate information about the relative spatial locality of states; that is, states

with the same h-value are assumed to be spatially local to each other in the state space. In this way, a semblance of structured ordering is imposed on the underlying hash tables[10]. Naturally, states with the same h-value are not guaranteed to be spatially local, but, as will be shown, this simple approach can be effective nonetheless.

An additional step that can be taken to reduce cache misses is to sort the successors by their h-values before performing duplicate detection. This ensures that all necessary operations on a sub-hash-table are performed before another must be accessed. The efficacy of this step depends on the size of the successor set, which in turn depends on the branching factor of the state space. When the branching factor is small enough, the cost of sorting the successor set is minimal, and so can be done without concern.

3.4.1 Experimental Comparison

The runtime performance of the h-value indexed array of hash tables is compared to a standard hash table[11] in two domains: the 15-puzzle domain introduced in Section 3.3.1 and a robot-arm trajectory planning problem.

Robot arm. The objective of the robot-arm trajectory planning problem [20, 19] is to find the minimum cost sequence of joint movements that manipulates the end of the arm, called the end-effector, from its initial location in a 50 by 50 grid to a desired location, while avoiding collisions with obstacles. There are both 6-degree-of-freedom (DoF) and 20-DoF versions of the problem,

[10]Note that this strategy could be used to potentially greater effect in algorithms that more explicitly leverage structure within the state space. For example, Sweep A* [33] exploits partial ordering within the state-space graph to expand states in layers. In this case, indexing the array by layer numbers would likely be more effective, as the spatial information is more concrete.

[11]The hash table implementation used is the phmap::flat_hash_map, which was modified from Google's Abseil flat_hash_map and can be found here: https://github.com/greg7mdp/parallel-hashmap.

and both versions can have uniform cost or non-uniform cost actions. The experiments here will use the 6-DoF version of the problem, which can be seen in Figure 3.4.

An action in this domain consists of incrementing or decrementing the angle of one of the arm's joints. In the unit-cost problem, all actions have unit-cost, regardless of the joint being manipulated. In the non-uniform cost problem, the cost of manipulating joint i is $N - i$, where $i \in [0, N-1]$, N is the number of joints, and $i = 0$ refers to the joint at the arm's base. The heuristic estimate used is the cost of the shortest-path from the end-effector's current grid cell to the goal cell, accounting for obstacles and assuming actions are unit-cost in the 8-connected grid.

Performance. Table 3.4 shows the speedup of hash table operations in the 15-puzzle domain and two versions of the robot-arm domain. These experiments were performed with A* using the standard two-level bucket queue. While the speedup for the 15-puzzle is a modest 10%, the speedup in the unit-cost and non-uniform cost 6-DoF robot arm versions is 30% and 41%, respectively. For added clarity, this amounts to a total time reduction of approximately 21 seconds for hash-table operations in the non-uniform cost robot arm problem. The primary reason that a greater improvement is achieved in the robot arm problems is that they have both a larger branching factor and state space. This increases the number of hash table accesses, as well as the size of the hash table. When the hash table grows larger, the likelihood of cache misses is increased. The modification to the hash table is intended to reduce the frequency of these cache misses. Given that this modification to hash-table behavior is trivial to implement, there is no reason not to use it for heuristic graph-search algorithms like A*.

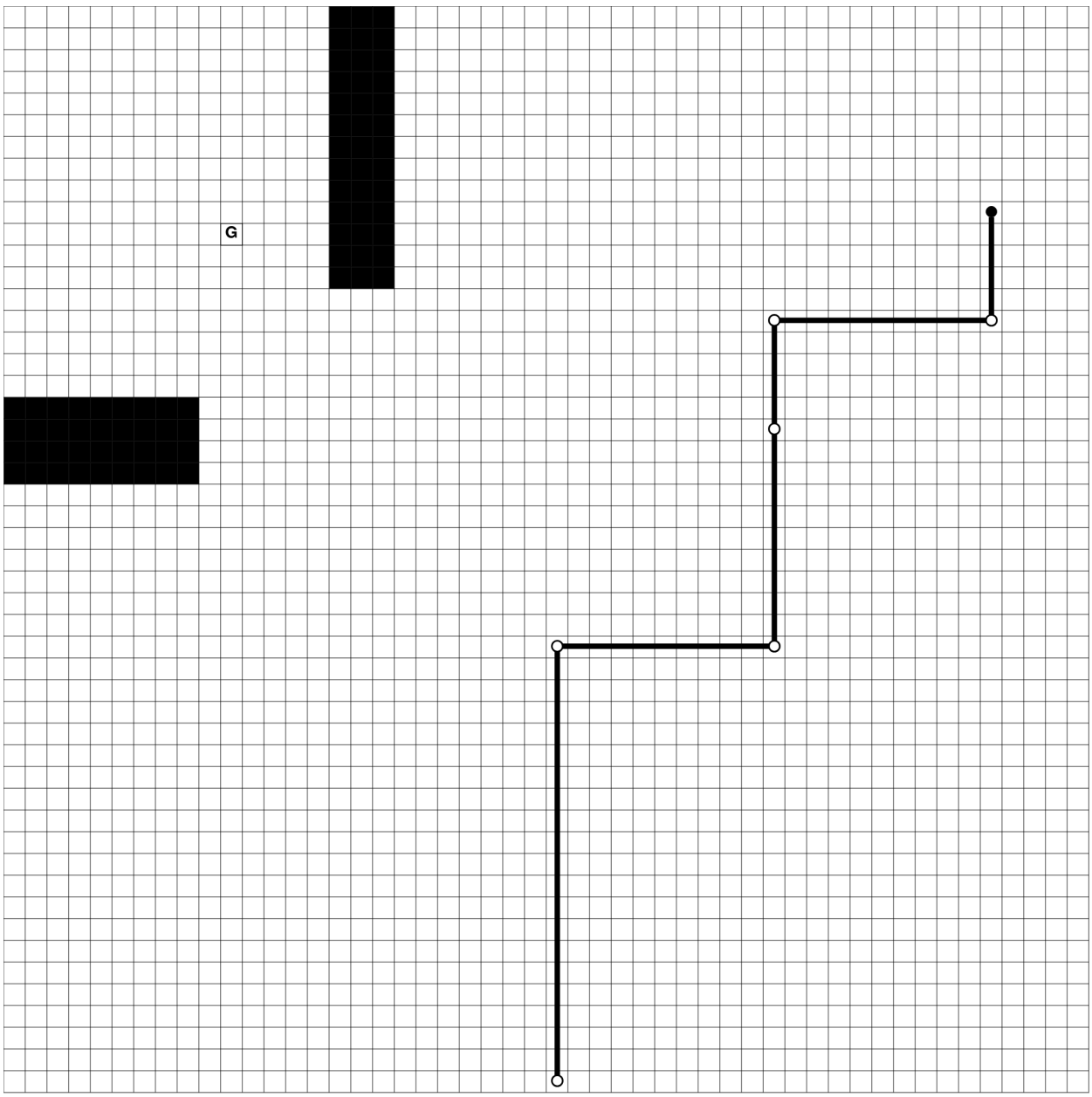


Figure 3.4: The initial configuration for the 6-DoF robot arm problem. Shaded cells represent obstacles in the environment and the goal location is marked by the letter 'G'.

Table 3.4: The speedup factor of hash table operations using the h-value indexed array of hash tables vs. a single hash table.

	Domain		
	15-puzzle	6-DoF Robot Arm (Unit-cost)	6-DoF Robot Arm (Non-uniform Cost)
Speedup	1.1x	1.3x	1.41x

CHAPTER 4

THE BUCKET HEAP

This chapter introduces a new bucket-based priority queue, called a bucket heap, that extends the performance benefits of the two-level bucket queue to bounded-cost, bounded-suboptimal, and anytime search algorithms. A detailed complexity analysis of the bucket heap is provided and its complexity is compared to an existing structure from the literature. Experiments are performed using several algorithms across a variety of domains to determine the practical efficacy of the bucket heap.

4.1 Structure

The two-level bucket queue and its variants discussed in Chapter 3 operate under the assumption that all states within a specific primary bucket will be expanded before any states in subsequent primary buckets are expanded. This makes them unsuitable for use with bounded-suboptimal, bounded-cost, or anytime search algorithms. The *bucket heap* is a hybrid data structure that overcomes this limitation, enabling improved priority-queue performance in a broader set of algorithms.

The first component of the bucket heap's structure is the modified two-level bucket queue, which functions exactly as described in Section 3.2. The second component is a binary heap of (indices to) the *non-empty* primary buckets in the modified two-level queue. This binary heap,

called the *primary heap*, orders the primary buckets by the search algorithm's evaluation function. Dequeue operations on the bucket heap are delegated to the primary bucket that is the current root of the primary heap, and when a primary bucket becomes empty, it is removed from the primary heap.

Evaluation function constraints. In order to efficiently compute the priorities associated with the primary buckets, a search algorithm's evaluation function must satisfy a specific constraint: from a set of states with identical f-values, the evaluation function must select states for expansion in non-decreasing order of h-value[1]. When this constraint is satisfied, the priority value of a primary bucket can be computed using an arbitrary state from its h_{min} secondary bucket. Thus, computing the priority value of a primary bucket is a constant-time operation, and the priority value only needs to be recomputed when h_{min} changes.

4.2 Complexity Analysis

This analysis will first examine the differences in search behavior that arise from bounded-suboptimal search compared to A* search; extrapolating to bounded-cost and anytime search follows naturally. In the context of a bounded-suboptimal search, one significant search property is affected: the range of f-values in the priority queue is increased. The number of primary buckets in the modified two-level queue must increase to accommodate this change. This behavior impacts the complexity of operations on the primary heap and, consequently, the bucket heap as a whole.

[1]Expansion could also be performed in decreasing order of h-value, but such an evaluation function seems unlikely to be useful in practice.

Theorem 2. *Let the variables π_0, C, and N_p denote the solution found by a BSS algorithm with suboptimality bound ϵ, the cost of the solution, $c(\pi_0)$, and the number of primary buckets in the bucket heap, respectively. $N_p \in O(C - f_{min})$ and, under appropriate assumptions, $N_p \in O(C^*)$.*

Proof. The cost of the solution returned by a bounded-suboptimal search algorithm is constrained such that $C \leq \epsilon \cdot f_{min}$. Given this, the largest f-value in the priority queue is at most $\epsilon \cdot f_{min} + T$. The total range of f-values is then $[f_{min}, \epsilon \cdot f_{min} + T]$, yielding a primary bucket count of $N_p = (\epsilon - 1) \cdot f_{min} + T$.

Assumption 1. The heuristic estimate used is admissible, and thus $f_{min} \in O(C^*)$.

Assumption 2. ϵ is a constant. Given that ϵ is a user-specified value, it is reasonable to assume that it is constant and therefore does not affect complexity.

Assumption 3. $T \in O(C^*)$. A domain in which the maximum transition cost is asymptotically greater than the optimal solution cost is inherently unsuited for the bucketing approach used by the bucket heap (and underlying two-level bucket queue). Domains exhibiting this behavior are therefore ignored for this analysis.

Under the above assumptions, the number of primary buckets in the bucket heap is $N_p \in O(C^*)$ when performing a bounded-suboptimal search. □

Theorem 2 characterizes the effect that using a BSS algorithm has on the number of primary buckets in the bucket heap. It is worth noting that these buckets may not all be non-empty (and often are not). Understanding the relationship between bounded-cost and bounded-suboptimal search [9], it is easy to see that framing a BCS algorithm in terms of its equivalent BSS representation

yields the same analysis. However, anytime search has an additional characteristic that impacts the performance of the bucket heap during a search; it does not terminate upon finding an initial solution. Consider an anytime algorithm that is allowed to run until it has proven an optimal solution. The algorithm will find a series of improved solutions, reducing its upper bound and increasing its lower bound until the two bounds converge, proving the current solution to be optimal. This anytime behavior, combined with pruning states where $f(s) \geq C$, naturally results in Corollary 2.1.

Corollary 2.1. *During an anytime search, the complexity of operations on the primary heap converges to constant time as the suboptimality bound converges to optimal.*

Similarly to the increased range of f-values, the range of h-values in the priority queue also increases when using a bounded-suboptimal, bounded-cost, or anytime search algorithm. Unlike the range of f-values, the range of h-values is not guaranteed to decrease during anytime search. However, it is clear that the maximum h-value in the priority queue cannot exceed the maximum f-value. This leads to Corollary 2.2.

Corollary 2.2. *Let N_s be the range of secondary buckets in the bucket heap. $N_s \in O(C^*)$.*

Having established the range of primary and secondary buckets in these algorithm classes, it's possible to analyze the complexity of individual bucket heap operations.

4.2.1 Enqueue

The Enqueue operation, like most of the bucket heap's operations, consists of two parts: delegation of the operation to the modified two-level queue and (possible) maintenance of the

primary heap. The procedure for an Enqueue operation is illustrated by Algorithm 4.1. Each algorithm presented for the bucket heap will make use of the previously defined algorithms for the two-level bucket queue and the binary heap from Sections 2.6 and 3.1, respectively.

The first step of the Enqueue operation is to insert the state into the modified two-level queue according to its previously established procedure. After this, any necessary changes to the primary heap must be handled. If the state was inserted into a previously empty primary bucket, this bucket must be inserted into the primary heap. Alternatively, if the state was inserted into a non-empty primary bucket and reduced its h_{min} value, a ChangeKey operation must be performed on that bucket in the primary heap. If neither scenario occurs, no additional work is done.

The complexity of the Enqueue operation depends on the complexity of the underlying data structures. The worst-case complexity of an Enqueue operation on the modified two-level bucket queue is $O(\log C^*)$. The complexity of both the Enqueue and ChangeKey operations on a binary heap are worst-case logarithmic in the number of elements; in this case, the number of non-empty primary buckets $\hat{N}_p$. By Theorem 2 the worst-case complexity of the bucket heap's Enqueue operation is then $O(\log C^*)$.

4.2.2 Dequeue

When performing the Dequeue operation, the state to be removed resides in the primary bucket at the root of the primary heap. A Dequeue is performed on this primary bucket and the state is returned. If this operation emptied the primary bucket, or altered its h_{min} reference, then the primary heap must reflect the change accordingly. If the primary bucket was emptied, then it is

Algorithm 4.1: Enqueue(A, H, s, f, h, pr)

Input: A, the modified two-level bucket queue
H, the array representing the primary heap
s, the state to enqueue
f, an operator that returns the f-value of a state
h, an operator that returns the h-value of a state
pr, an operator that returns the priority of a primary bucket

```
1 Enqueue(A, s, f, h) // Algorithm 3.1
2 i ← f(s) mod |A|
3 if |A[i]| = 1 then
4 |   Enqueue(H, f(s), pr) // Algorithm 2.3
5 else if min_{A[i]} changed then
6 |   ChangeKey(H, f(s), pr) // Algorithm 2.7
```

dequeued from the primary heap. Instead, if h_{min} was altered, then a ChangeKey operation is performed on this primary bucket in the primary heap. See Algorithm 4.2 for details.

As with the Enqueue operation, the complexity of the Dequeue operation depends on the composing data structures. The Dequeue operation on the modified two-level queue has a worst-case complexity of $O(\log C^*)$. The Dequeue and ChangeKey operations on the primary heap are, again, logarithmic in the number of non-empty primary buckets. The worst-case complexity of the bucket heap's Dequeue operation is then the same as the Enqueue operation, $O(\log C^*)$.

4.2.3 ChangeKey

The ChangeKey operation follows a similar pattern to the previous operations, but requires more care to ensure that the primary heap is properly maintained. Because a ChangeKey operation in the modified two-level bucket queue is effectively a Dequeue followed by an Enqueue, the conditions for updating the primary heap in both of those operations must be considered. This can be seen in Algorithm 4.3. First, the ChangeKey is performed on the modified two-level queue, updating the

Algorithm 4.2: Dequeue(A, H, pr)

```
Input: A, the modified two-level bucket queue
       H, the primary heap
       pr, an operator that returns the priority of a primary bucket
1 B ← A[H[0] mod |A|]
2 s ← Dequeue(B) // Algorithm 2.12
3 if B = ∅ then
4 |   Dequeue(H, pr) // Algorithm 2.5
5 else if min_B changed then
6 |   ChangeKey(H, 0, pr) // Algorithm 2.7
7 return s
```

state's location. Then the primary heap must be managed for the primary bucket in which the state resided originally, as well as its current bucket.

Reflecting the steps of the Dequeue operation, if the original primary bucket was emptied, a Dequeue removes it from the primary heap. If, instead, its h_{min} value was altered, a ChangeKey operation is performed on the primary heap. When neither condition is met, no additional work is done. If the state was moved into a previously empty primary bucket, that bucket is Enqueued into the primary heap. Or, if the primary bucket's h_{min} value was reduced, a ChangeKey occurs on that bucket in the primary heap. Again, if neither occurs, no extra work is done.

The worst-case complexity of the modified two-level bucket queue's ChangeKey operation is $O(\log C^*)$, and the primary heap operations are $\Theta(\log \hat{N}_p)$. So, as with the Enqueue and Dequeue operations, the bucket heap's ChangeKey operation has a worst-case complexity of $O(\log C^*)$.

4.2.4 LowerBound and Reorder

The LowerBound and Reorder operations of the bucket heap are trivially implemented, so pseudocode is omitted. Despite their simplicity, these operations are incredibly advantageous

Algorithm 4.3: ChangeKey$(A, H, s, f, h, pr, \hat{f})$

Input: A, the modified two-level bucket queue
H, the primary heap
s, a reference to the existing state in the queue
f, an operator that returns the f-value of a state
h, an operator that returns the h-value of a state
pr, an operator that returns the priority of a primary bucket
$\hat{f}$, the old f-value of s

```
B_1 ← A[f̂ mod |A|]
B_2 ← A[f(s) mod |A|]
ChangeKey(A, s, f, h, f̂) // Algorithm 3.3
// Handle edge cases for the primary and secondary buckets that s
   was moved from
if B_1 = ∅ then
    Dequeue(H, f̂, pr) // Algorithm 2.5, modified to support removal from
       an arbitrary location
else if min_{B_1} changed then
    ChangeKey(H, f̂, pr)
// Handle edge cases for the primary and secondary buckets that s
   was moved to
if |B_2| = 1 then
    Enqueue(H, f(s), pr)
else if min_{B_2} changed then
    ChangeKey(H, f(s), pr)
```

for algorithms that maintain suboptimality bounds or have evaluation functions that may alter during search. Because the modified two-level bucket queue maintains a reference to f_{min}, the LowerBound operation is $\Theta(1)$. To efficiently perform the Reorder operation, the bucket heap exploits the structure of the modified two-level bucket queue to avoid recomputing the priority values of all states in the queue. Instead, the priorities of each bucket in the primary heap are recomputed, which takes constant time per bucket. The total cost of computing these priorities is $\Theta(\hat{N}_p)$. After the priority values are computed, a Reorder is performed on the primary heap, which is $\Theta(\hat{N}_p)$. So, by Theorem 2 the worst-case complexity of a Reorder operation on the bucket heap is $O(C^*)$.

4.2.5 Memory

Just as the complexity of operations on the bucket heap depend on the composing data structures, so too does the memory complexity. The memory overhead of the modified two-level queue is given by the number of primary buckets multiplied by the number of secondary buckets, which is $O(C^{*2})$ in the worst case. The overhead of the primary heap is $O(C^*)$ in the worst case, and so the memory overhead of the bucket heap structure itself is $O(C^{*2})$. The total memory complexity of the bucket heap is then $O(n + C^{*2})$, where n is the maximum number of states in the queue simultaneously. The overhead of the bucket heap itself drops from the analysis when n is the dominant term.

4.3 Related Work

The only bucket-based priority queue in the literature that attempts to extend beyond the A* algorithm is an unnamed structure from Gilon *et al.* that is briefly described in their paper

introducing dynamic potential search [9]. The authors' objective with this structure was to mitigate the cost of the Reorder operation when the state evaluation function changed during the search, a key operation for their algorithm. However, they did not attempt to generalize the structure beyond this algorithm, nor did they suggest the possibility. Furthermore, neither a complexity analysis nor an experimental analysis of the priority queue's performance was provided, and the authors did not discuss any benefits that might be gained from the structure other than for their Reorder operation. Their follow-up paper ([10]) does not mention the structure.

4.3.1 Structure and Operation Complexities

Unfortunately, the authors do not provide much detail about their implementation. They describe their structure as a priority queue of (g, h) pairs from the two-level bucket queue described in [2]. This is determined to be a binary heap that maintains what this document has called secondary buckets. It is not explicitly stated whether empty secondary buckets are removed from this binary heap; for this discussion, it will be assumed that this is the case. There are two primary differences in approach from the bucket heap: first, this structure is built upon the original two-level bucket queue (Section 3.1), and second, the binary heap operates on non-empty secondary buckets, instead of non-empty primary buckets.

Because the necessary foundation for a complexity analysis of this structure was already provided during the analysis of the bucket heap, a detailed analysis of each operation will not be provided. Instead, Table 4.1 shows the worst-case complexities of all operations. The implications of these complexities will be discussed in the next section, where the bucket heap will be compared to both the binary heap and to this structure.

Table 4.1: The worst-case complexities of operations for each of the priority queues.

	Worst-Case Complexity				
	Enqueue	Dequeue	ChangeKey	LowerBound	Reorder
Binary Heap	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(n)$	$\Theta(n)$
Bucket Heap	$O(\log C^*)$	$O(\log C^*)$	$O(\log C^*)$	$\Theta(1)$	$O(C^*)$
Gilon Structure	$O(\log C^*)$	$O(C^*)$	$O(C^*)$	$\Theta(1)$	$O(C^{*2})$

4.4 Theoretical Comparison

This section will discuss the theoretical advantages that the bucket heap provides compared to the binary heap and the structure described by Gilon *et al.* For ease of reference, Table 4.1 shows the worst-case complexities of each operation for the binary heap, the bucket heap, and the Gilon structure.

The first major distinction between operations on the bucket heap and the Gilon structure is for the Dequeue and ChangeKey operations. The bucket heap has an exponentially better worst-case complexity for these operations because it is built upon the modified two-level bucket queue introduced in Section 3.2, and by trivially extending Lemma 2 to the primary heap, it is clear that the bucket heap cannot be asymptotically worse than the binary heap. However, as was shown in the example in Figure 3.2, this is not the case for the two-level bucket queue and, by extension, the Gilon structure.

The second major difference between the bucket heap and the Gilon structure is the complexity of the Reorder operation. Although both of these priority queues outperform the binary heap in this case, the bucket heap performs the operation in linear time compared to the quadratic time of the Gilon structure. This is because the bucket heap's primary heap only operates on non-empty primary buckets, whereas the binary heap in the Gilon structure operates on all non-

empty secondary buckets. Applying Theorem 2 and Corollary 2.2 yields this worst-case quadratic complexity for the Gilon structure.

The final consideration between the bucket heap and the Gilon structure is not visible within the worst-case operation complexities. Consider the complexity of the Enqueue operation for both structures in Table 4.1; they are the same. However, it is expected that the Gilon structure will take twice as long to perform this operation in practice, because the number of non-empty secondary buckets is quadratic. This is not relevant for the asymptotic comparison, but is a distinguishing characteristic nonetheless. In fact, when considering the expected performance of these two structures in an anytime algorithm, the difference in bucket count is increasingly relevant.

Lemma 4. *During an anytime search, Reorder operations on the Gilon structure can cost C^* times more than Reorder operations on the bucket heap.*

Proof. As stated in Corollary 2.1, operations on the bucket heap's primary heap converge to constant time during an anytime search. By Corollary 2.2, the secondary bucket range remains $O(C^*)$. Thus, in the last stages of anytime search, the worst-case Reorder cost is $\Theta(1)$ for the bucket heap and $O(C^*)$ for the Gilon structure. □

Table 4.2 shows the the complexity of operations on the binary heaps of buckets for both the bucket heap and the Gilon structure. This gives some insight into how the different bucketing strategies affect the expected performance of these two bucket-based priority queues on average. Examining these complexities from this perspective emphasizes the performance advantage inherent in operating on non-empty primary buckets instead of non-empty secondary buckets. Consider a scenario in which all operations on the underlying two-level bucket queue variants are constant-

Table 4.2: The complexities of operations on the binary heaps of buckets for the bucket heap and the Gilon structure. $\hat{N}_p$ is the number of non-empty primary buckets, $\hat{N}_s$ is the number of non-empty secondary buckets.

	Worst-Case Complexity			
	Enqueue	Dequeue	ChangeKey	Reorder
Bucket Heap	$\Theta(\log \hat{N}_p)$	$\Theta(\log \hat{N}_p)$	$\Theta(\log \hat{N}_p)$	$\Theta(\hat{N}_p)$
Gilon Structure	$\Theta(\log \hat{N}_s)$	$\Theta(\log \hat{N}_s)$	$\Theta(\log \hat{N}_s)$	$\Theta(\hat{N}_s)$

time. The comparison between the bucket heap and the Gilon structure then depends directly on the difference between the number of non-empty primary and secondary buckets, which identifies some interesting attributes.

Lemma 5. *Let n, $\hat{N}_s$, and $\hat{N}_p$ be the number of states in the queue, the number of non-empty secondary buckets, and the number of non-empty primary buckets, respectively. The following relationship holds: $n \in \Omega(\hat{N}_s) \in \Omega(\hat{N}_p)$.*

Proof. By Theorem 2, the number of non-empty secondary buckets is at most equal to the number of states in the queue. Applying the same logic, a primary bucket is non-empty iff it contains one or more non-empty secondary buckets. Thus, at worst, $\hat{N}_p = \hat{N}_s$. □

Corollary 5.1. *$\hat{N}_p = \hat{N}_s$ iff each non-empty primary bucket contains exactly one non-empty secondary bucket.*

By Lemma 5, it is not possible for operations on the bucket heap's primary heap to exceed the cost of an equivalent operation on the Gilon structure's binary heap. Furthermore, Corollary 5.1 makes it clear that, in general, these operations are unlikely to have equivalent cost, especially in the context of anytime search (Corollary 2.1).

4.5 Experiments

Having discussed the theoretical characteristics of the bucket heap, an experimental analysis will be performed. Several algorithms are tested in a variety of domains to explore the effectiveness of the bucket heap under different conditions, and to compare its performance to the binary heap and the structure from Gilon *et al.*

4.5.1 Test Algorithms

To highlight the flexibility of the bucket heap, several algorithms will be tested and their performance compared when implemented with a bucket heap, a binary heap, or with the structure from Gilon *et al.* In particular, two bounded-suboptimal algorithms will be used, together with their corresponding anytime algorithms (because of their equivalence with BSS algorithms, BCS algorithms are omitted [9]).

Optimistic Search and Bounded Anytime Weighted A*. The first pair of algorithms tested are *Optimistic Search* (OPS) and its anytime extension, *Bounded Anytime Weighted A** (BAWA*) [28]. Optimistic Search begins by performing Weighted A* search, using the weighted state evaluation function:

$$f_w(s) = g(s) + w \cdot h(s)$$

The weight w is set by inflating the desired suboptimality bound by a user-determined factor. For these experiments, $w = 1 + 2 * (\epsilon - 1)$, as defined in [28]. The intuition behind scaling the weight is that Weighted A* search often returns a solution that is much better than the suboptimality bound, thus causing it to do more work than necessary. In addition to a binary heap ordered by

the weighted evaluation function $f_w(s)$, Optimistic Search also maintains a second binary heap ordered by f-value. This allows the algorithm to determine the suboptimality of a solution as $\frac{C}{f_{min}}$, the tightest possible suboptimality bound [12]. If the initial solution is proven to be within the desired bound, the algorithm terminates. Otherwise, the algorithm begins expanding states in order of f-value in an attempt to improve the bound on solution quality and prove that the solution is within the desired bound. However, if there exists an f_w-value in the priority queue that is less than the cost of the current solution, that state is expanded first. This process continues until a solution is proven to be within the desired suboptimality bound.

Bounded Anytime Weighted A* is the natural extension of Optimistic Search into an anytime algorithm. Rather than terminating once a solution is proven to be within a desired bound (though it could be used for that, too), BAWA* is allowed to continue executing until it either proves that a solution is optimal or it exceeds a predetermined time limit. This algorithm is nearly identical to Anytime Weighted A* [11, 12], except that it maintains the second binary heap ordered by f-value, giving it access to the ideal suboptimality bound throughout the search. Both OPS and BAWA* are able to prune states where $f(s) \geq C$, reducing the size of the priority queue.

Dynamic Potential Search and Anytime Non-Parametric A*. The second pair of algorithms tested are *Dynamic Potential Search* (DPS)[9] and *Anytime Non-Parametric A** (ANA*) [29], also called Anytime Potential Search [25]. Dynamic Potential Search expands states in maximal order of the *potential function*:

$$u(s) = \frac{\epsilon \cdot f_{min} - g(s)}{h(s)}$$

This function's dependence on f_{min} has two implications for DPS: first, the algorithm must have access to f_{min}, and second, when the value of f_{min} changes, the evaluation function must reflect this change, requiring a Reorder operation. These two issues prompted the structure from Gilon *et al.* [9]. It is also possible to implement the algorithm by maintaining two binary heaps, one ordered by f-value and the other by u-value. DPS terminates upon finding its first solution, as its evaluation function guarantees that the solution is within the desired suboptimality bound.

Anytime Non-Parametric A* also expands states in maximal order of the potential function, but the function is modified from that of DPS:

$$u(s) = \frac{C - g(s)}{h(s)}$$

Because it depends on the current solution cost, this potential function cannot be used until an initial solution is discovered, which can be done using any desired method. Here, the initial solution will be found using greedy heuristic search, as was done in [25]. Dependence on the current solution cost also means that the evaluation function must be updated when an improved solution is found, requiring a Reorder operation. The bound on solution suboptimality is provided by the minimal u-value that has been expanded so far; this bound is not as tight as $\frac{C}{f_{min}}$ but does not require knowledge of f_{min}. ANA* can terminate when it has either proven that its solution achieves a desired suboptimality bound or it has exceeded a predetermined time limit. ANA* also prunes states where $f(s) \geq C$.

It may not be immediately clear that the potential functions used by DPS and ANA* satisfy the necessary conditions for use with the bucket heap. By replacing the $g(s)$ term in the potential

function of ANA* with $f(s) - h(s)$, the function can be simplified to the following, equivalent potential function:

$$u(s) = \frac{C - f(s)}{h(s)} + 1$$

This rewritten function makes it obvious that for a set of states with equivalent f-values, the state with minimal h-value is preferred, satisfying the condition necessary to be used with the bucket heap. The potential function for DPS can be rewritten similarly and so is omitted for brevity.

4.5.2 Test Domains

Experimental data was gathered from four domains: the 15-puzzle, a robot arm trajectory planning problem, path planning in a two-dimensional grid, and multiple sequence alignment. This section will define each domain where necessary, explain how data was collected from each domain, and describe the configuration of each of the test algorithms for each domain.

15-Puzzle. The 15-puzzle domain is identical to the description in Section 3.3.1. However, for these experiments, all 100 of Korf's 15-puzzle instances were used to gather the data, which was averaged across the instances. Both optimistic search and dynamic potential search were configured to find solutions with a suboptimality bound of $\epsilon = 1.05$. Anytime non-parametric A* and bounded anytime weighted A* were allowed to execute until their solutions were proven to be optimal, and the weight for BAWA* was set to $w = 1.5$.

Robot arm trajectory planning. Data was gathered for both the unit-cost and non-uniform cost versions of the 6-DoF robot arm problem, described in Section 3.4.1. For both cost models, DPS and OPS were configured with a desired suboptimality bound of $\epsilon = 1.02$. Again, ANA* and

BAWA* were executed until their solutions were proven to be optimal. BAWA* was configured with the weights $w = 1.25$ and $w = 3.0$ for the unit-cost and non-uniform cost versions of the problem, respectively.

Two-Dimensional grid navigation. The objective of this domain is to find the shortest sequence of actions that navigates from the grid's top left corner to its bottom right corner, while avoiding obstacles. The action set corresponds to the cardinal directions (north, south, east, and west), and each action moves to the next grid cell in the specified direction if it does not contain an obstacle. All actions are unit cost, and the heuristic used is the Manhattan distance. Data was gathered from experiments on three sizes of square grid, with side lengths of 500, 2500, and 5000. For each grid size, 25 random instances were generated with obstacles occupying 20% of the grid cells, and the data was averaged across these instances. OPS and DPS were set with a suboptimality bound of $\epsilon = 1.015$, and ANA* and BAWA* were allowed to converge to optimal solutions. BAWA* was executed with the weight $w = 1.025$.

Multiple sequence alignment. The alignment of protein and DNA sequences is a common task in computational biology and bioinformatics [3] and is used to compute similarity metrics between the sequences. Figure 4.1 shows an example of a partial alignment of eight DNA sequences from different species [25]. Each row of the output alignment corresponds to a particular sequence, and each column in a row is assigned either the next character in its sequence or a *gap*. The cost of the alignment is computed using the sum-of-pairs. For all $\binom{k}{2}$ pairs of sequences in the output, the cost of the pairwise alignment is computed and added to the total. The cost of a pairwise alignment is determined by comparing the characters in each column of the pair and assigning

them a cost. Identical characters receive a *match* cost, differing characters receive a *mismatch* cost, and a character aligned to a gap receives a gap cost. The cost values used for a match, mismatch, and gap were taken from [25] and are 0, 2, and 1, respectively[2].

Multiple sequence alignment (MSA) can be framed in terms of finding a shortest path in a hyperdimensional lattice, where the dimension of the lattice d is determined by the number of sequences [3, 15, 32]. The total size of the state space is $O(l^d)$, where l is the length of the longest sequence. A state can be represented by a d-dimensional coordinate c, where each entry c_i is an index in the corresponding sequence, $i \in [0, d-1]$. An action is a d-dimensional offset o, where each entry o_i is zero or one, $i \in [0, d-1]$. A one represents selecting the associated character from the sequence to be placed at this column in the final alignment, whereas a zero constitutes inserting a gap into the column. The branching factor of MSA is then $2^d - 1$ (the identity offset is ignored). The heuristic used is the sum of optimal pairwise alignments. These optimal alignments were computed via dynamic programming prior to execution of the search algorithms.

Six protein sequences from [15] were used to test this domain[3]. Data was gathered from alignments of three, four, five, and six sequences. For alignments of k protein sequences, data was averaged across the $\binom{6}{k}$ combinations of sequences. For all alignments, OPS and DPS were configured with suboptimality bounds of $\epsilon = 1.02$. For alignments of three, four, and five sequences, ANA* and BAWA* were allowed to converge to optimal solutions, and BAWA* was given a weight of $w = 1.1$. However, for aligning all six sequences, ANA* and BAWA* were given desired suboptimality bounds of $\epsilon = 1.02$, and BAWA* was set with the weight $w = 1.03$.

[2]In biological applications, it is common to use a matrix that represents evolutionary similarities between different amino acids to compute the cost of aligning two individual amino acids [16, 31]

[3]Protein sequences were accessed from https://www.ncbi.nlm.nih.gov/.

Scarites	C	T	T	A	G	A	T	C	G	T	A	C	C	A	A	–	–	–	A	A	T	A	T	T	A	C
Carenum	C	T	T	A	G	A	T	C	G	T	A	C	C	A	C	A	–	T	A	C	–	T	T	T	A	C
Pasimachus	A	T	T	A	G	A	T	C	G	T	A	C	C	A	C	T	A	T	A	A	G	T	T	T	A	C
Pheropsophus	C	T	T	A	G	A	T	C	G	T	T	C	C	A	C	–	–	–	A	C	A	T	A	T	A	C
Brachinus armiger	A	T	T	A	G	A	T	C	G	T	A	C	C	A	C	–	–	–	A	T	A	T	A	T	T	C
Brachinus hirsutus	A	T	T	A	G	A	T	C	G	T	A	C	C	A	C	–	–	–	A	T	A	T	A	T	A	C
Aptinus	C	T	T	A	G	A	T	C	G	T	A	C	C	A	C	–	–	–	A	C	A	A	T	T	A	C
Pseudomorpha	C	T	T	A	G	A	T	C	G	T	A	C	C	–	–	–	–	–	A	C	A	A	A	T	A	C

Figure 4.1: An example alignment of eight DNA sequences [25].

4.5.3 Data analysis

Although the data from the experiments is organized according to domain, discussion and interpretation of the results will be done collectively, with the exception of the two-dimensional grid and multiple sequence alignment results, which will be discussed last. Performance data for each combination of domain and algorithm is given in terms of the speedup factor for each priority queue operation, computed as $\frac{t_o}{\hat{t}_o}$, where t_o is the time taken by the operation in the binary heap implementation and $\hat{t}_o$ is the time taken for the operation in the implementation identified by the table entry. This allows for a detailed comparison of the relative performance for each priority queue implementation and makes it easy to see which operations are affected by the different characteristics of the domains and algorithms. This data is shown in Tables 4.3–4.13, but Tables 4.3–4.5, corresponding to the 15-puzzle, unit-cost, and non-uniform-cost 6-DoF robot arm domains, will be considered first. When an algorithm does not use a particular operation, or that operation was unused in the given domain, the entry in the table is marked as not applicable (N/A). At a high level, a few things are immediately clear from these tables: both bucket-based priority queues generally outperform the binary heap implementations for every operation, the bucket heap consistently outperforms the Gilon structure, and the speedup factors for the bounded-suboptimal

algorithms are higher than for the anytime algorithms. What is not as clear is which characteristics of the algorithms and domains impact these results.

Algorithm performance differences. The most important attribute that affects the relative performance of the priority queues is the size of the priority queue itself; that is, the number of states present in the queue. This follows directly from the asymptotic complexities of the data structures and explains why the bounded-suboptimal algorithms have higher speedup factors. Because the bounded-suboptimal algorithms typically guarantee that the first solution found is within the desired suboptimality bound, they are unable to make effective use of pruning to reduce the size of the priority queue, and so the bucket-based priority queues provide larger speedups over the binary heap implementations. In fact, as $\epsilon \rightarrow 1.0$, it becomes more difficult to prove that a solution achieves the desired suboptimality, further inflating the size of the priority queue and increasing the expected speedup factors from using a bucket-based priority queue. This is intuitive because $\epsilon = 1.0$ is equivalent to A* search.

By design, optimistic search does not have to guarantee that the initial solution achieves the desired suboptimality, instead relying on the ideal bound $\frac{C}{f_{min}}$ to prove it afterward. This allows OPS to use pruning to some effect and explains why the speedup factors are larger for DPS—it has the largest priority queue. Additionally, both DPS and OPS require the use of two priority queues when implemented using the binary heap, increasing the priority queue overhead and thus the speedup factors when compared to the anytime algorithms.

The performance differences between anytime non-parametric A* and bounded anytime weighted A* also stem from the core issue of priority queue size. This is affected by the way these algorithms

find an initial solution and subsequently work to improve it or prove its optimality. In the case of the 15-puzzle (Table 4.3), BAWA* converges to the optimal solution faster than ANA*, on average. This is because a weight of $w = 1.5$ is known to be highly effective in this domain [12, 25]. The initial solution discovered by BAWA* in this domain is typically optimal (or nearly so), and the rest of the algorithm's runtime is spent proving it. This enables BAWA* to maximize its pruning, minimizing the size of the priority queue, resulting in a smaller speedup factor for the bucket-based structures. In contrast, the initial solution discovered by ANA* can be very poor in this domain, causing it to make less effective use of pruning during the earlier stages of the search. Furthermore, by using the ideal suboptimality bound, BAWA* can detect convergence to an optimal solution more quickly than ANA*, reducing the total number of priority queue operations performed. For these reasons, higher speedup factors are achieved for ANA* than BAWA* in the 15-puzzle domain. This behavior does not hold in the robot arm domain (Tables 4.4 and 4.5), where BAWA* encounters some of the issues associated with using a fixed weight that are explored in [19] and [25].

Operation performance differences. In general, the Enqueue operation has the smallest speedup factor for each algorithm and domain. This is consistent with the expected behavior of these data structures because the average-case complexity of the Enqueue operation on a binary heap is $\Theta(1)$. In addition to maintaining their binary heaps, the two bucket-based priority queues must also delegate to their underlying two-level bucket queues. This may require resizing a dynamic array if the target secondary bucket is full; using a geometric growth strategy amortizes these resizes to constant time, but constant factors still affect practical performance. The efficiency of

delegating to the two-level bucket queue is also affected by the relationship between the domain's cost function, heuristic, and branching factor. When a state is expanded, a series of Enqueue operations is performed on its successors. The number of combinations of f- and h-values for these successors affects the likelihood of cache misses during the Enqueue operations, which can degrade performance. The fact that these data structures still provide speedup for the Enqueue operation is a testament to the effectiveness of bucketing as a strategy in heuristic search.

Also consistent with expected behavior is that the ChangeKey operation has the next-smallest speedup factors, after the Enqueue operation. The average-case complexity of the ChangeKey operation on a binary heap is $\Theta(\log n)$, which is why the speedup is larger than for the Enqueue operation. However, ChangeKey operations in the two bucket-based priority queues have more overhead than their Enqueue operations. Recall that the ChangeKey operation in the two-level bucket queue—and therefore the bucket heap and the Gilon structure—corresponds to a Dequeue followed by an Enqueue. Not only does this incur the cost of both operations on the underlying two-level bucket queues, but each operation may also require manipulation of the accompanying binary heap. Lastly, because the Dequeue is performed on an arbitrary secondary bucket, it is far more likely to experience a cache miss. This combination of factors makes ChangeKey the least efficient operation for the two bucket-based structures, and it only provides a larger speedup factor than the Enqueue operation because of the binary heap's average-case constant complexity for Enqueue.

The Dequeue operation has the largest speedup factor of the operations used by all test algorithms. Like the ChangeKey operation, the Dequeue operation on a binary heap is $\Theta(\log n)$ on average, so it is expected that significant speedup can be achieved by the bucketing approaches.

Because the Dequeue operation only removes data from the priority queue, it never requires a memory allocation. In general, the Dequeue operation is also very cache-friendly because a standard Dequeue (one not part of ChangeKey) is always performed on a secondary bucket that has recently been accessed by either a Dequeue, Enqueue, or Reorder operation. The remaining costs are then maintaining h_{min}, which was shown to be efficient in practice in Section 3.3.1, and maintaining the associated binary heap, which is the same as for the other operations. This makes Dequeue the most practically efficient operation on the two bucket-based priority queues, excluding Reorder.

Finally, the Reorder operation, for applicable algorithms, provides incredible speedup factors over the binary heap implementations. The reasoning is simple: there can be exponentially more states in the priority queue than there are non-empty buckets in the two bucket-based structures, and as always, the larger the priority queue, the greater the speedup. However, the massive speedup of the Reorder operation raises the question of why the total priority queue speedup factors are not greater. In short, the answer is diminishing returns. Suppose that the Reorder operation accounted for 90% of the priority queue overhead in the binary heap implementation of an algorithm. Amdahl's law [1] can be adapted to compute the maximum possible priority queue speedup achievable by improving only the Reorder operation:

$$\frac{1}{1 - P + \frac{P}{S}}$$

where P is the percentage of priority queue overhead attributed to the Reorder operation, and S is the speedup factor provided for Reorder. In this case, it is never possible to achieve a total speedup greater than 10. The Gilon structure was proposed in [9] to reduce the cost of Reordering in DPS,

but it is clear that focusing on improvement of a single operation is ultimately insufficient. In fact, for DPS, the total priority queue speedup achievable by focusing on the Reorder operation is fairly minimal. For the bucket-based priority queues to achieve the highest speedup of the Reorder operation, it is necessary for the size of the priority queue to be exponentially larger than the number of non-empty buckets. Because the number of Reorder operations that can occur in DPS is bounded by the number of primary buckets, this means that there are exponentially more of the other priority queue operations. For a binary heap, if there are d Dequeue operations of cost $\log n$, r Reorder operations of cost n, and n is exponentially greater than r, it follows that Reordering can account for at most 50% of total priority queue overhead. By Amdahl's law, eliminating the cost of Reordering entirely can provide at most a factor of 2 priority queue speedup in dynamic potential search.

Bucket heap vs. Gilon structure. Although both bucket-based priority queues outperform the binary heap, the bucket heap consistently outperforms the structure from Gilon *et al.* In the 15-puzzle domain, the bucket heap is typically twice as fast. This behavior was predicted by the theoretical analysis in Section 4.4 and is explained by the fact that the number of non-empty secondary buckets can be quadratically more than the number of non-empty primary buckets. The number of operations on the binary heap within the Gilon structure is also expected to be more, as it must be maintained whenever any secondary bucket becomes empty or non-empty. In contrast, the bucket heap only needs to maintain its binary heap when an h_{min} value is changed.

The fact that the bucket heap is less than twice as fast in the unit-cost version of the 6-DoF robot arm domain, and equally as fast in the non-uniform-cost version, are artifacts of unique

Table 4.3: The speedup factor of priority queue operations for each algorithm relative to their implementations using a binary heap (or two, where necessary), averaged across Korf's 100 15-puzzle instances.

		Algorithm							
		OPS		BAWA*		DPS		ANA*	
	Operation	Bucket Heap	Gilon *et al.*	Bucket Heap	Gilon *et al.*	Bucket Heap	Gilon *et al.*	Bucket Heap	Gilon *et al.*
Speedup	Enqueue	**2.82x**	1.52x	**2.28x**	1.0x	**3.37x**	1.74x	**2.41x**	0.80x
	Dequeue	**9.56x**	4.67x	**4.10x**	2.11x	**10.84x**	4.92x	**5.08x**	1.46x
	ChangeKey	**6.47x**	4.13x	**4.78x**	3.53x	**7.32x**	4.49x	**4.12x**	2.24x
	Reorder	N/A	N/A	N/A	N/A	**>151,045.0x**	57,871.65x	**2,473.04x**	693.29x
	Total	**5.79x**	3.03x	**3.31x**	1.60x	**6.82x**	3.37x	**3.82x**	1.20x

Table 4.4: The speedup factor of priority queue operations for each algorithm relative to their implementations using a binary heap (or two, where necessary) in the 6-DoF unit-cost robot arm domain.

		Algorithm							
		OPS		BAWA*		DPS		ANA*	
	Operation	Bucket Heap	Gilon *et al.*	Bucket Heap	Gilon *et al.*	Bucket Heap	Gilon *et al.*	Bucket Heap	Gilon *et al.*
Speedup	Enqueue	**1.90x**	1.76x	**1.91x**	1.49x	**2.50x**	2.34x	**1.67x**	1.16x
	Dequeue	**14.75x**	9.61x	**10.38x**	7.10x	**19.29x**	11.95x	**9.13x**	5.41x
	ChangeKey	**4.32x**	1.90x	**4.76x**	3.03x	**5.12x**	2.20x	**1.96x**	1.18x
	Reorder	N/A	N/A	N/A	N/A	**>233,067.0x**	9,711.13x	**226.25x**	21.05x
	Total	**6.11x**	4.55x	**5.88x**	4.23x	**9.0x**	6.55x	**4.95x**	3.16x

Table 4.5: The speedup factor of priority queue operations for each algorithm relative to their implementations using a binary heap (or two, where necessary) in the 6-DoF non-uniform cost robot arm domain.

		Algorithm							
		OPS		BAWA*		DPS		ANA*	
	Operation	Bucket Heap	Gilon *et al.*	Bucket Heap	Gilon *et al.*	Bucket Heap	Gilon *et al.*	Bucket Heap	Gilon *et al.*
Speedup	Enqueue	**1.52x**	1.50x	1.65x	**1.75**	**1.67x**	1.66x	**1.31x**	1.27x
	Dequeue	**24.64x**	22.61x	**17.19x**	16.15x	**29.28x**	27.37x	**18.17x**	17.01x
	ChangeKey	**2.49x**	2.26x	**3.21x**	2.96x	**2.43x**	2.39x	**1.32x**	1.20x
	Reorder	N/A	N/A	N/A	N/A	**216,690.37x**	6,050.30x	**3,821.11x**	639.12x
	Total	**10.06x**	9.67x	8.51x	**8.53x**	**14.34x**	13.93x	**8.80x**	8.39x

characteristics of the domains themselves. The same heuristic is used for both cost models, and the range of values for the heuristic is quite small. The heuristic is more accurate in the unit-cost domain, so the small range of values only has the effect of modestly reducing the performance difference between these two priority queues. However, in the non-uniform cost domain, the heuristic is highly uninformed. This results in the number of non-empty primary and non-empty secondary buckets being nearly equal, largely mitigating the performance difference between the two structures, and in the specific case of BAWA*, a slight performance advantage for the Gilon structure. This advantage comes from its improvement of the Enqueue operation, which is possible because the constant factors of the Enqueue operation in the bucket heap are slightly higher. This increase in constant-factor overhead is mostly due to additional branching, which handles edge cases in maintaining the binary heap. Misprediction of these branches is typically not a concern, but the combination of BAWA* and the 6-DoF non-uniform-cost robot arm domain is a worst-case scenario for comparing the bucket heap to the Gilon structure. Despite this, the performance difference remains negligible, and the bucket heap still outperforms the Gilon structure for the other algorithms in this domain.

Grid navigation and multiple sequence alignment. Path-planning in a two-dimensional grid and multiple sequence alignment are two examples of worst-case domains for comparing the bucket-based priority queues to the binary heap, and so they have been saved for last. Understanding which characteristics of these two domains cause this worst-case behavior is important for understanding the applicability of bucket-based approaches to heuristic search as a whole. The results for two-dimensional grid navigation shown in Tables 4.6–4.9 are in stark contrast to the results from the

domains discussed previously. In particular, the structure from Gilon *et al.* is outperformed by the binary heap implementations in every case—including the Reorder operation! The bucket heap manages to provide speedup for the Enqueue and Dequeue operations in most cases, albeit very minimal, and still provides substantial speedup for the Reorder operation[4]. However, the bucket heap does not improve the ChangeKey operation in any case.

The significant disparity between the bucket heap and the Gilon structure, most specifically for the Reorder operation, highlights a key factor that impacts the effectiveness of bucketing in this domain: the ratio of states to buckets. Recall that the number of non-empty secondary buckets in the two-level bucket queue can never exceed the number of states in the queue. Asymptotically, it is not possible for the Reorder operation on the Gilon structure to be worse than the same operation on the binary heap, but the results here show it performing worse in practice. The first reason for this is that the Manhattan distance heuristic is highly accurate, causing the number of states in the priority queue to be relatively small; or, more importantly, the ratio of states to non-empty secondary buckets is close to 1.0. The second factor also depends on the heuristic, specifically the range of h-values, which ultimately determines the range of secondary buckets.

For a square grid of side length l, the initial heuristic estimate is $h_0 = 2 * l - 2$, given the start and goal locations defined previously. This is an extensive range of secondary buckets, and unlike the states within a binary heap, these secondary buckets are not all contiguous in memory. This results in poor cache locality during the Reorder operation of the Gilon structure in this domain, degrading its performance to be worse than the binary heap. The bucket heap avoids this level of performance degradation because it only operates on non-empty primary buckets when Reordering, which is

[4]The Reorder operation for DPS is marked N/A in Table 4.8 because the algorithm did not need to Reorder to achieve a solution within the desired suboptimality bound.

more cache-friendly[5]. The increased range of secondary buckets also exacerbates the propensity for cache misses during the ChangeKey operation of the bucket-based structures, which is why the bucket heap does not provide speedup for the operation. In general, the more clever design of the bucket heap better mitigates many of the issues presented by this worst-case domain.

Because multiple sequence alignment is equivalent to path-planning in a hyper-dimensional lattice, it presents some of the same issues as path-planning in a two-dimensional grid. However, Tables 4.10–4.13 show that as the number of sequences aligned increases, the bucket-based priority queues become increasingly advantageous. Similar to the two-dimensional grid, the bucket heap is not as adversely affected by the characteristics of MSA, even when aligning just a few sequences. Incrementing the number of aligned sequences increases the branching factor exponentially, resulting in a more favorable ratio of states to buckets, improving the general performance of the bucketing strategies. Additionally, the accuracy of the heuristic is reduced as the number of sequences increases, contributing to a larger number of states being present in the priority queue. The trend shown across all domains is that the bucket heap is the more effective bucket-based priority queue and is the best replacement for the binary heap in bounded-suboptimal, bounded-cost, and anytime algorithms to which it is applicable.

[5]Conceptually, the range of primary buckets could expand to a degree that causes substantial degradation to the bucket heap; however, because the range of primary buckets converges to 1 during anytime search, the degradation would be temporary.

Table 4.6: The speedup factor of priority queue operations for OPS relative to the implementation using two binary heaps in two-dimensional grid navigation.

		500x500		2500x2500		5000x5000	
	Operation	Bucket Heap	Gilon *et al.*	Bucket Heap	Gilon *et al.*	Bucket Heap	Gilon *et al.*
Speedup	Enqueue	**1.23x**	0.37x	**1.10x**	0.25x	0.95x	0.23x
	Dequeue	**2.04x**	0.45x	**1.57x**	0.32x	**1.49x**	0.31x
	ChangeKey	0.82x	0.24x	0.74x	0.19x	0.67x	0.20x
	Total	**1.52x**	0.39x	**1.33x**	0.29x	**1.23x**	0.28x

Table 4.7: The speedup factor of priority queue operations for BAWA* relative to the implementation using two binary heaps in two-dimensional grid navigation.

		500x500		2500x2500		5000x5000	
	Operation	Bucket Heap	Gilon *et al.*	Bucket Heap	Gilon *et al.*	Bucket Heap	Gilon *et al.*
Speedup	Enqueue	**1.43x**	0.41x	**1.16x**	0.27x	**1.02x**	0.24x
	Dequeue	**2.07x**	0.47x	**1.67x**	0.33x	**1.60x**	0.31x
	ChangeKey	0.89x	0.27x	0.73x	0.20x	0.66x	0.20x
	Total	**1.67x**	0.43x	**1.41x**	0.30x	**1.32x**	0.28x

Table 4.8: The speedup factor of priority queue operations for DPS relative to the implementation using two binary heaps in two-dimensional grid navigation.

		500x500		2500x2500		5000x5000	
	Operation	Bucket Heap	Gilon *et al.*	Bucket Heap	Gilon *et al.*	Bucket Heap	Gilon *et al.*
Speedup	Enqueue	**1.26x**	0.37x	**1.28x**	0.28x	**1.16x**	0.27x
	Dequeue	**2.19x**	0.51x	**1.82x**	0.39x	**1.76x**	0.38x
	ChangeKey	0.83x	0.25x	0.77x	0.20x	0.69x	0.21x
	Reorder	N/A	N/A	N/A	N/A	N/A	N/A
	Total	**1.62x**	0.43x	**1.55x**	0.34x	**1.48x**	0.33x

Table 4.9: The speedup factor of priority queue operations for ANA* relative to the implementation using a binary heap in two-dimensional grid navigation.

		500x500		2500x2500		5000x5000	
	Operation	Bucket Heap	Gilon *et al.*	Bucket Heap	Gilon *et al.*	Bucket Heap	Gilon *et al.*
Speedup	Enqueue	**1.14x**	0.30x	**1.23x**	0.28x	**1.06x**	0.26x
	Dequeue	**1.58x**	0.37x	**1.57x**	0.34x	**1.63x**	0.35x
	ChangeKey	0.60x	0.18x	0.56x	0.18x	0.47x	0.17x
	Reorder	**17.1x**	0.47x	**18.21x**	0.30	**27.57x**	0.29x
	Total	**1.31x**	0.33x	**1.40x**	0.31x	**1.36x**	0.31x

Table 4.10: The speedup factor of priority queue operations for OPS relative to the implementation using two binary heaps in multiple sequence alignment.

		Sequences Aligned							
		3		4		5		6	
	Operation	Bucket Heap	Gilon *et al.*	Bucket Heap	Gilon *et al.*	Bucket Heap	Gilon *et al.*	Bucket Heap	Gilon *et al.*
Speedup	Enqueue	0.94x	0.44x	0.83x	0.45x	**1.54x**	1.28x	**1.83x**	1.70x
	Dequeue	**2.87x**	0.54x	**2.42x**	0.53x	**8.88x**	6.55x	**15.92x**	14.17x
	ChangeKey	**1.23x**	0.55x	0.69x	0.40x	**1.59x**	1.46x	**1.84x**	1.74x
	Total	**1.27x**	0.48x	**1.05x**	0.47x	**3.18x**	2.61x	**3.97x**	3.68x

Table 4.11: The speedup factor of priority queue operations for BAWA* relative to the implementation using two binary heaps in multiple sequence alignment.

		Sequences Aligned							
		3		4		5		6	
	Operation	Bucket Heap	Gilon *et al.*	Bucket Heap	Gilon *et al.*	Bucket Heap	Gilon *et al.*	Bucket Heap	Gilon *et al.*
Speedup	Enqueue	0.88x	0.40x	**1.02x**	0.59x	**1.51x**	1.02x	**1.59x**	1.34x
	Dequeue	**2.44x**	0.81x	**5.82x**	2.50x	**19.71x**	10.94x	**22.72x**	9.72x
	ChangeKey	0.93x	0.43x	**1.31x**	0.68x	**2.87x**	1.98x	**2.02x**	1.51x
	Total	**1.62x**	0.62x	**3.06x**	1.52x	**7.82x**	4.96x	**5.11x**	3.57x

Table 4.12: The speedup factor of priority queue operations for DPS relative to the implementation using two binary heaps in multiple sequence alignment.

		Sequences Aligned							
		3		4		5		6	
	Operation	Bucket Heap	Gilon *et al.*	Bucket Heap	Gilon *et al.*	Bucket Heap	Gilon *et al.*	Bucket Heap	Gilon *et al.*
Speedup	Enqueue	**1.06x**	0.53x	**1.05x**	0.58x	**1.73x**	1.26x	**1.70x**	1.40x
	Dequeue	**3.36x**	0.73x	**3.99x**	0.98x	**15.4x**	5.93x	**25.26x**	9.72x
	ChangeKey	**1.27x**	0.59x	**1.12x**	0.59x	**2.36x**	1.51x	**2.03x**	1.52x
	Reorder	**>5.2x**	2.42x	**1,221.67x**	2.45x	**12,009.03x**	28.62x	**55,162.75x**	85.43x
	Total	**1.53x**	0.61x	**1.82x**	0.77x	**5.65x**	3.28x	**5.98x**	4.05x

Table 4.13: The speedup factor of priority queue operations for ANA* relative to the implementation using a binary heap in multiple sequence alignment.

		Sequences Aligned							
		3		4		5		6	
	Operation	Bucket Heap	Gilon *et al.*	Bucket Heap	Gilon *et al.*	Bucket Heap	Gilon *et al.*	Bucket Heap	Gilon *et al.*
Speedup	Enqueue	0.91x	0.35x	**1.06x**	0.57x	**1.59x**	1.14x	**1.69x**	1.39x
	Dequeue	**3.10x**	0.75x	**7.97x**	2.76x	**21.23x**	9.99x	**22.21x**	8.49x
	ChangeKey	0.67x	0.32x	**1.30x**	0.60x	**2.33x**	1.52x	**2.03x**	1.51x
	Reorder	**25.52x**	1.12x	**47.03x**	1.15x	**315.74x**	9.55x	**80.43x**	2.27x
	Total	**1.64x**	0.54x	**3.85x**	1.66x	**7.78x**	4.67x	**4.87x**	3.30x

CHAPTER 5

ALGORITHMIC IMPROVEMENTS ENABLED BY THE BUCKET HEAP

This chapter discusses the benefits of using a bucket heap that extend beyond improving the performance of priority queue operations. The instant access to f_{min} provided by the bucket heap facilitates several algorithmic improvements: access to the ideal suboptimality bound, flexible modification of the evaluation function during search, improved worst-case complexity through a hybrid approach, and tighter suboptimality bounds in difficult domains.

5.1 Computing a Better Suboptimality Bound

Access to the suboptimality bound $\frac{C}{f_{min}}$ is advantageous for anytime search algorithms. This suboptimality bound provides more accurate information about the quality of a returned solution and allows an algorithm to terminate earlier by recognizing sooner when its solution achieves the desired suboptimality bound.

In [4] it was shown that ANA* can be considered a special case of *anytime focal search*, which is itself an extension of *focal search* [23]. In focal search, which falls into the category of bounded-suboptimal algorithms, a FOCAL list is maintained in addition to the OPEN list, where FOCAL is a subset of OPEN[1]. Anytime focal search generalizes this to include typical anytime properties, such as pruning, anytime termination, and return of a bound on solution quality. When

[1]In some cases, FOCAL may not need to be maintained explicitly. Weighted A* is an example of a focal search algorithm that does not maintain a separate FOCAL list [9]

implementing ANA* as an anytime focal search, the OPEN list is ordered by f-value and FOCAL is ordered by the potential function.

Two specific examples will be used repeatedly throughout this Chapter: instance 100 from Korf's 15-puzzle instances and the non-uniform-cost 6-DoF robot arm. Experimental data will be presented in terms of state expansions, rather than execution time. This makes it easier to see how the discussed techniques affect the algorithms, independent of any change in priority queue performance. It should be clear by now that implementing the discussed techniques will be more efficient with a bucket heap compared to using two binary heaps, and so emphasis is given to the algorithmic improvement itself. Figures 5.1 and 5.2 show the suboptimality bounds per expansion for ANA* and focal ANA* on the 15-puzzle instance and the 6-DoF robot arm, respectively. The focal version of ANA* is able to achieve substantially better suboptimality bounds with fewer expansions, corroborating the results presented in [4]. When an anytime algorithm is implemented using the bucket heap, access to this tighter suboptimality bound is provided without any additional overhead, inherently improving the algorithm's anytime characteristics.

Some algorithms make use of the ideal suboptimality bound to modify the evaluation function during search; these algorithms can gain even more benefit from using the bucket heap. It is well known that anytime algorithms that use a fixed weight in their evaluation functions, such as anytime weighted A* and bounded anytime weighted A*, may perform poorly in certain domains [20, 19, 25]. Algorithms have been proposed that attempt to address this issue directly by decreasing the weight during the search in a structure manner. One method that is considered particularly

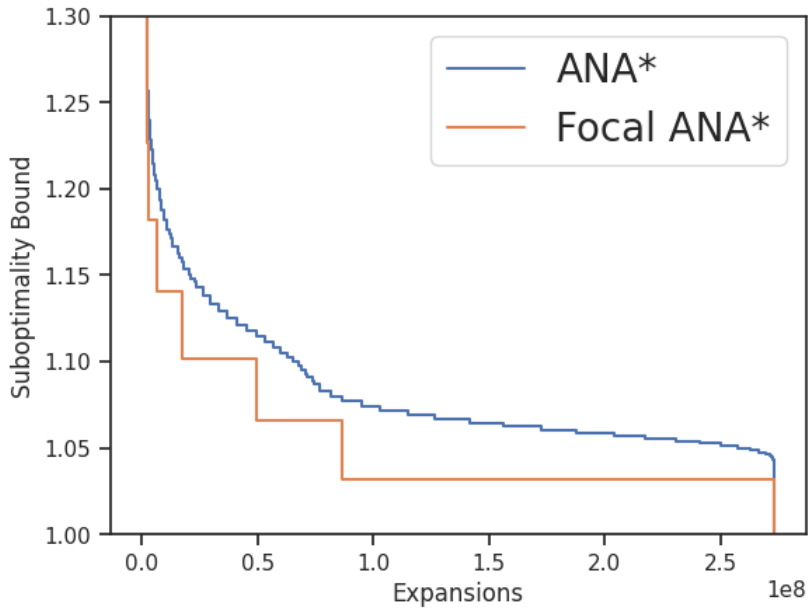


Figure 5.1: The suboptimality bounds per expansion for ANA* and Focal ANA* on instance 100 of the 15-puzzle.

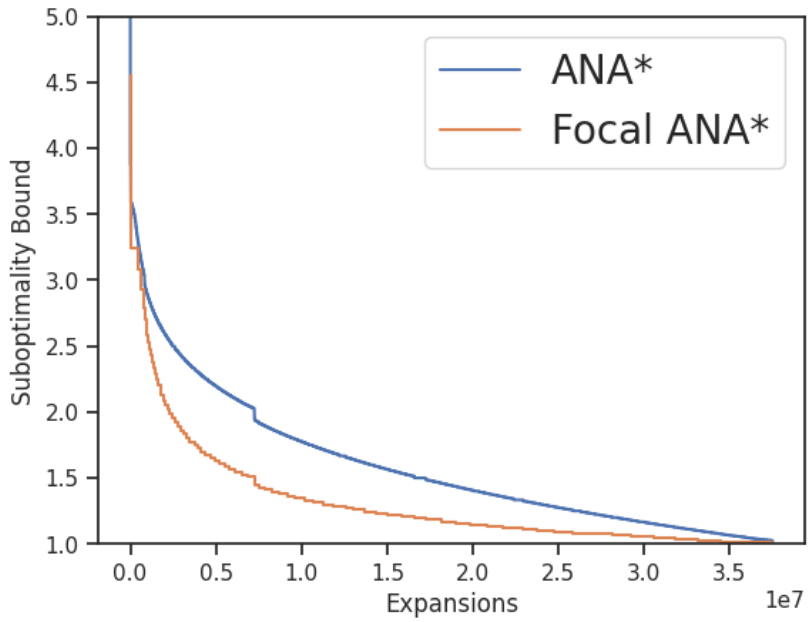


Figure 5.2: The suboptimality bounds per expansion for ANA* and Focal ANA* on the non-uniform-cost 6-DoF robot arm.

effective is to set the weight equal to the current suboptimality bound[2] [27]. This creates the following evaluation function:

$$\hat{f}_w(s) = g(s) + \frac{C}{f_{min}} \cdot h(s).$$

The intuition behind this approach is that by setting the weight equal to the suboptimality bound, the algorithm emphasizes decreasing the suboptimality bound in an attempt to provide a better bound at any time during the search. The drawback is that, where the fixed-weight evaluation function $f_w(s)$ depends only on the weight, which is determined a priori, this updated evaluation function has dependencies on both the upper and lower bounds. This requires both knowledge of f_{min} and that the priority queue be Reordered whenever the upper bound or lower bound changes. Because the standard approach makes use of two binary heaps, this evaluation function can incur significant overhead, requiring more Reorder operations than either ANA* or DPS. However, the cost of Reordering with the bucket heap is negligible, and so the increased Reorder frequency caused by this evaluation function can be managed without adversely affecting performance.

The evaluation function $\hat{f}_w(s)$ is sometimes used as part of more complex search algorithms; for the sake of simplicity, it will be tested independently of any other techniques. To establish the initial upper bound, greedy search will be used, as was the case for ANA*. Figures 5.3 and 5.4 plot the suboptimality bounds per expansion as before, including the algorithm defined by $\hat{f}_w(s)$. The evaluation function $\hat{f}_w(s)$ performs as expected, more effectively reducing the suboptimality bound throughout the search. This is particularly noticeable in Figure 5.4, where during the early stages of the search it is able to provide the same suboptimalty bound as focal ANA* in half as

[2]This method often includes subtracting a small user-provided value, typically denoted ϵ, from this weight. To avoid confusion from notation conflicts, this term has been omitted for this discussion.

many state expansions. Because the potential function of ANA* primarily focuses on improving the upper bound, it can struggle to effectively improve its suboptimality bound when the current solution is optimal. This is because the only way to prove that a solution is optimal is to have the upper and lower bounds converge, which does not happen as quickly if the upper bound is the sole focus. However, there is a far more important drawback to neglecting the lower bound: state re-expansions.

5.2 Limiting State Re-Expansions

It is well established that when A* search is combined with an inconsistent heuristic it may perform an exponential number of state re-expansions in the worst case [21]. The state evaluation functions of the bounded-suboptimal and anytime algorithms discussed so far are inadmissible and, therefore, inconsistent. These algorithms are then subject to this same worst-case number of state re-expansions, even when the underlying heuristic upon which they are built is consistent. Here, a simple and unique strategy is proposed that eliminates this issue.

Theorem 3. *For bounded-suboptimal and anytime search algorithms that build upon A* with a consistent heuristic, a hybrid algorithm can be constructed that performs the same number of expansions as A*, asymptotically*[3].

Proof. Let E_{A*} represent the number of expansions performed by A* to solve a given problem. Let $\hat{f}(s)$ be an inadmissible evaluation function constructed using the same consistent heuristic as A*, and let k be a user-defined constant. Consider the properties of an algorithm that expands states in the order prescribed by $\hat{f}(s)$, but every kth expansion is performed according to $f(s)$. The

[3]It is likely that this approach can be generalized to work with any inconsistent evaluation function, though such a generalization may not be particularly useful.

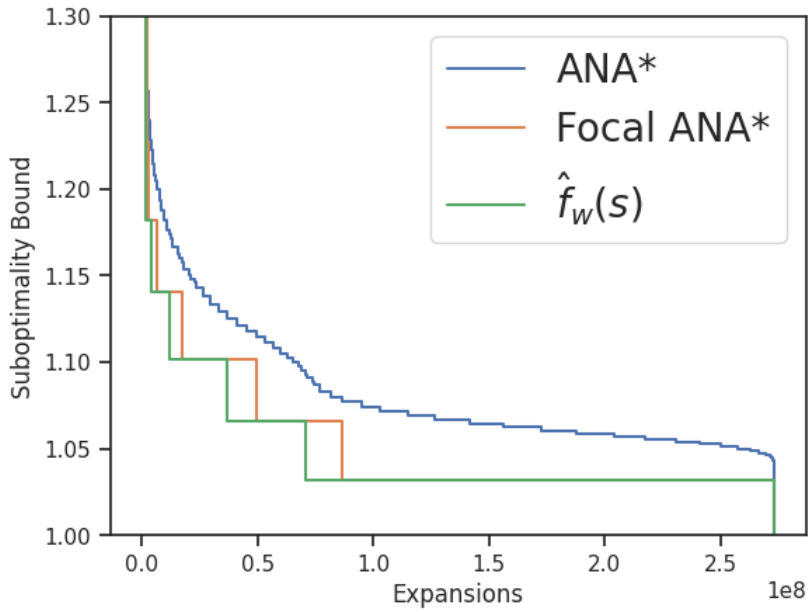


Figure 5.3: The suboptimality bounds per expansion for ANA*, Focal ANA*, and $\hat{f}_w(s)$ on instance 100 of the 15-puzzle.

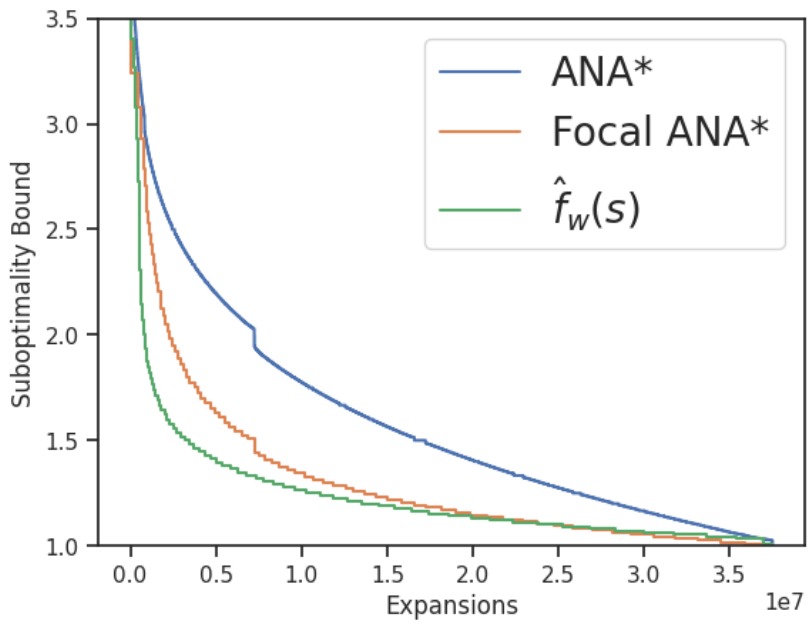


Figure 5.4: The suboptimality bounds per expansion for ANA*, Focal ANA*, and $\hat{f}_w(s)$ on the non-uniform-cost 6-DoF robot arm.

algorithm described is a hybrid of A* and the algorithm defined by $\hat{f}(s)$. Once E_{A*} expansions are performed according to $f(s)$, it is guaranteed that the search must be complete. Because expansions are performed according to $\hat{f}(s)$ k times as frequently, the total number of expansions performed by the hybrid algorithm is $O(k \cdot E_{A*})$. Since k is a constant, this hybrid approach is guaranteed to perform the same number of expansions as A*, asymptotically. □

This theoretical result is very useful for guaranteeing that any excessively poor worst-case behavior is avoided, no matter how ineffective the inconsistent evaluation function $\hat{f}(s)$ may be. Because it depends on the ability to expand states in order of both f-value and another evaluation function, the traditional approach would be to use two binary heaps to implement this strategy. It should be clear that using a bucket heap is a more effective way to construct this hybrid algorithm, assuming that the evaluation function $\hat{f}(s)$ satisfies the necessary criteria to be compatible with the bucket heap. To explore the practical advantages of this technique, the *n-pancake problem* will be used [8], along with the two-dimensional grid domain used previously.

The n-pancake problem consists of a stack of n pancakes, each with a unique diameter in the range $[1, n]$. Actions are unit-cost and constitute flipping a substack of pancakes, reversing their order. The initial configuration of the stack is arbitrary, and the objective is to manipulate the stack until it is sorted by diameter in non-decreasing order, with the smallest pancake at the top of the stack, using the fewest actions. The n-pancake problem is equivalent to finding the minimal number of prefix reversals required to sort a set of integers and is NP-Hard. The *gap* heuristic is used [14], where a gap is defined by a pancake at index i having a diameter that differs from the

diameter of the pancake at index $i + 1$ by more than 1, and the heuristic estimate is the current number of gaps present[4].

Table 5.1 shows the ratio of state expansions relative to A* for three different algorithms, along with the expansion ratio for the hybrid versions of each algorithm. For simplicity, the hybrid algorithms expand in order of f-value every other expansion, so their expansion ratio cannot exceed 2.0, although it can be less, as the data shows. Expansion data was collected and averaged from 100 random instances of the 48-pancake problem. The weight used for both versions of BAWA* was $w = 1.25$. BAWA* performs more than 11 times as many expansions as A* in this domain. Naturally, the expansion ratio for this algorithm is affected by the weight selection, where a higher weight would result in an even larger ratio and setting $w = 1.0$ would perform identically to A*. This highlights one of the disadvantages of a fixed-weight algorithm, because it is a trade-off between more rapid solution discovery and the potential for an excessively high number of state expansions. The hybrid version of this algorithm provides a significant improvement, performing only 8% more expansions than A*, which is much better than the worst-case guarantee provided by the hybrid strategy.

The other two algorithms are specifically designed to avoid the issues of a fixed-weight approach and, in this case, manage to do so reasonably well. However, ANA* and the algorithm ordered by $\hat{f}_w(s)$ still perform 64% and 36% more expansions than A*, respectively, which is not insignificant. Their hybrid implementations almost completely eliminate these additional expansions, expanding only 6% and 3% more states than A*. Reducing the total number of state expansions an algorithm performs directly reduces the computational overhead of the algorithm, so even this seemingly

[4]To avoid the heuristic considering an inverted stack to be a goal state, the domain often includes a ”plate” of diameter $n + 1$ that cannot be moved from the bottom of the stack but is included in gap calculations.

Table 5.1: Expansion ratios relative to A* for the 48-pancake problem.

	Default			Hybrid		
	ANA*	BAWA*	$\hat{f}_w(s)$	ANA*	BAWA*	$\hat{f}_w(s)$
Ratio	1.64x	11.73x	1.36x	1.06x	1.08x	1.03x

modest reduction is beneficial. Furthermore, these more adaptive algorithms can still experience behavior similar to that of BAWA*, depending on the characteristics of the domain.

Table 5.2 shows the expansion ratios relative to A* for the three algorithms in the two-dimensional grid navigation domain from Chapter 4. Like before, data was collected and averaged over 25 random instances for each of the grid sizes, with obstacles occupying 20% of the grid cells. The weight used for BAWA* and its hybrid implementation was $w = 1.025$. This domain emphasizes the significant impact that an inconsistent evaluation function can have on the performance of an algorithm. Because of the small weight used, BAWA* performs better than the other two algorithms, but expanding more than 14 times as many states as A* in the largest grid size can hardly be considered effective. The algorithm ordered by $\hat{f}_w(s)$ performs in excess of 30 times more expansions than A*, and ANA* is even worse with a factor greater than 57! The hybrid implementations of each algorithm have expansion ratios less than the guaranteed factor of 2. Considering that expansion count relates directly to computation time, this simple hybrid approach results in a 30x speedup for ANA* in the largest grid size! The hybrid strategy has strong theoretical and practical implications for bounding the number of state expansions an algorithm performs, but it also has additional advantages beyond this.

Table 5.2: Expansion ratios relative to A* for the different sizes of two-dimensional grid.

		Default			Hybrid		
	Grid Size	ANA*	BAWA*	$\hat{f}_w(s)$	ANA*	BAWA*	$\hat{f}_w(s)$
Ratio	500x500	5.85x	1.70x	3.41x	1.53x	1.33x	1.37x
	2500x2500	28.35x	7.15x	14.23x	1.84x	1.72x	1.70x
	5000x5000	57.59x	14.63x	30.49x	1.90x	1.82x	1.77x

5.3 Achieving Tighter Suboptimality Bounds

A natural consequence of the hybrid strategy presented in the previous section is that anytime algorithms implemented in this way will not be subject to neglect of the lower bound. Figures 5.3 and 5.4 showed that the evaluation function $\hat{f}_w(s)$ generally achieves tighter suboptimality bounds during search because it focuses more specifically on this improvement, while ANA* focuses primarily on reducing the upper bound, ignoring the lower bound. Figures 5.5 and 5.6 again plot the suboptimality bounds per expansion as was done in Section 5.1, this time omitting ANA* but including hybrid implementations of the other two algorithms. The most important thing to note is that the hybrid version of focal ANA* achieves the best suboptimality bound at all times during the search in both domains. Surprisingly, the hybrid strategy using $\hat{f}_w(s)$ performs quite poorly in both domains, especially in the early stages of the search. An explanation of this behavior will be provided shortly.

For some domains, proving reasonable suboptimality bounds is exceedingly difficult, either because of the size of the state space or because the heuristic is highly uninformed. In domains like this, the hybrid strategy can not only provide better suboptimality bounds but may be crucial to allowing an algorithm to provide a meaningful suboptimality bound at all before a system's memory capacity is reached. Two particularly difficult domains will be used to explore this behavior: the

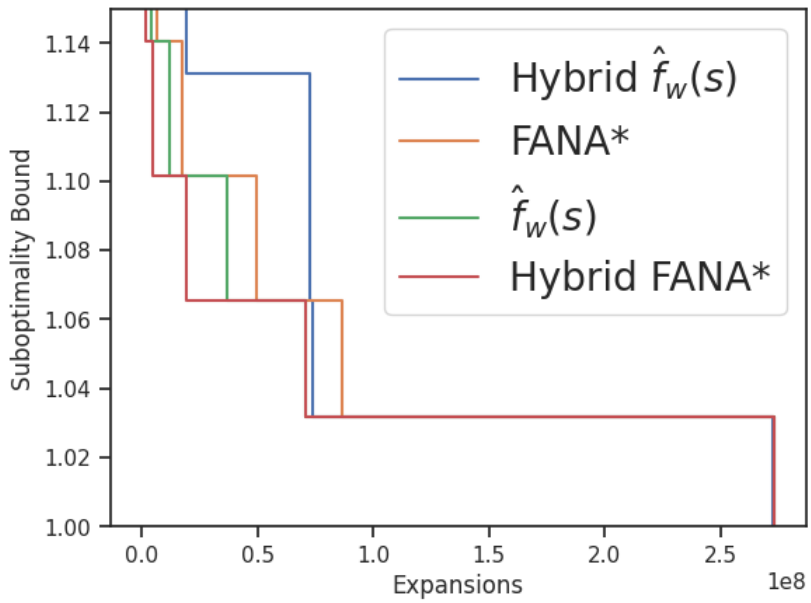


Figure 5.5: The suboptimality bounds per expansion for Focal ANA*, $\hat{f}_w(s)$, and their hybrid implementations on instance 100 of the 15-puzzle.

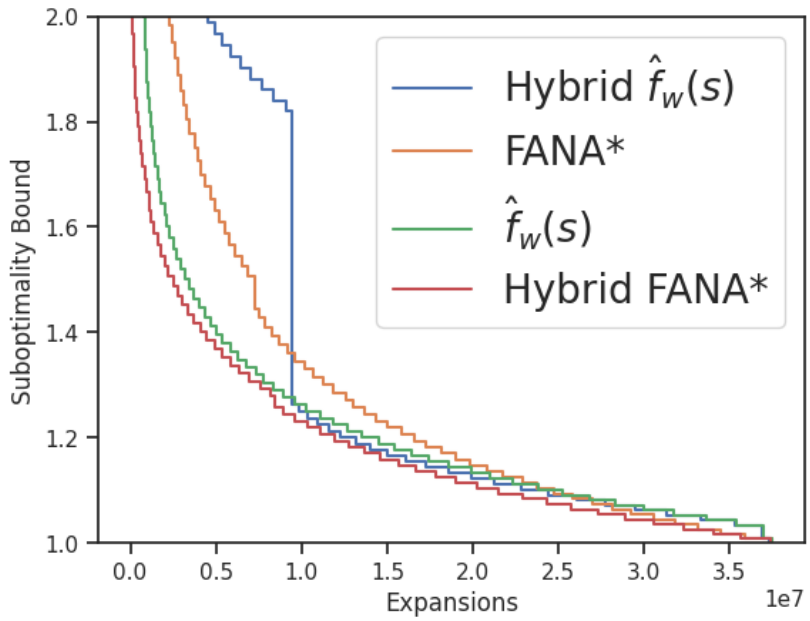


Figure 5.6: The suboptimality bounds per expansion for Focal ANA*, $\hat{f}_w(s)$, and their hybrid implementations on the non-uniform-cost 6-DoF robot arm.

non-uniform-cost 20-DoF robot arm and the *heavy n-pancake problem* [9]. The heavy n-pancake problem differs from the original in that its actions are not unit cost. Instead, the cost of flipping a substack is the maximum between the diameters of the pancake at the top of the stack and the flip position. For these experiments, the gap heuristic is used, unmodified, resulting in a highly uninformed heuristic, greatly increasing the difficuly of the problem.

Heavy 16-pancake problem. Table 5.3 shows the average suboptimality bounds and solution costs achieved in 5 million expansions by focal ANA*, $\hat{f}_w(s)$, and hybrid implementations of each for the heavy 16-pancake problem. The data was averaged over 25 random instances. Unsurprisingly, the hybrid version of focal ANA* achieves a better suboptimality bound than focal ANA* alone, and provides the best suboptimality bound overall. This behavior matches that seen in Figures 5.3 and 5.4. However, just as before, the hybrid implementation using $\hat{f}_w(s)$ provides a worse suboptimality bound on average than the other algorithms. This is because the evaluation function $\hat{f}_w(s)$ already attempts to minimize the suboptimality bound, which can prevent it from improving the upper bound as effectively as ANA*. The hybrid algorithm exacerbates this behavior because half of all expansions specifically target improvement of the lower bound, causing it to neglect the upper bound further. The average solution costs of the algorithms make this behavior quite clear, with the solution costs provided by the hybrid $\hat{f}_w(s)$ algorithm being much higher than those provided by the other algorithms. Figure 5.7 shows the suboptimality bounds per expansion on an arbitrary single instance from the set of heavy 16-pancake problems solved. Interestingly, the hybrid algorithm using $\hat{f}_w(s)$ provides the tightest suboptimality bound throughout the search

Table 5.3: Suboptimality bounds and solution costs achieved in 5 million expansions for the heavy 16-pancake problem.

	Default		Hybrid	
	Focal ANA*	$\hat{f}_w(s)$	Focal ANA*	$\hat{f}_w(s)$
ϵ	3.90	3.08	2.57	3.96
C	169.24	196.92	169.32	266.44

for this instance. Focal ANA* provides the worst bound by far in this case, due to its neglect of the lower bound.

Despite the fact that Theorem 3 guarantees that the hybrid strategy performs the same number of expansions as A*, asymptotically, regardless of the paired evaluation function, the practical characteristics of the hybrid algorithm depend heavily on the paired evaluation function. Care must be taken to select an evaluation function that works in tandem with the hybrid strategy to minimize the number of ineffective expansions performed. Because the hybrid version of focal ANA* provides the best suboptimality bound, it seems that the most effective approach to improving the suboptimality bound may be this combination of evaluation functions that alternately focus on improving the upper bound and the lower bound. This is intuitive given that the ideal suboptimality bound is the ratio of the upper and lower bounds, and therefore improvement to either bound necessarily improves the suboptimality bound. However, the suboptimality bounds in Figure 5.7 act as a counterpoint to this supposition. Furthermore, instead of alternating with every expansion, introducing a level of introspection to determine when states should be expanded in order of f-value would likely improve practical performance, especially if that introspection accounts for characteristics of the alternate evaluation function.

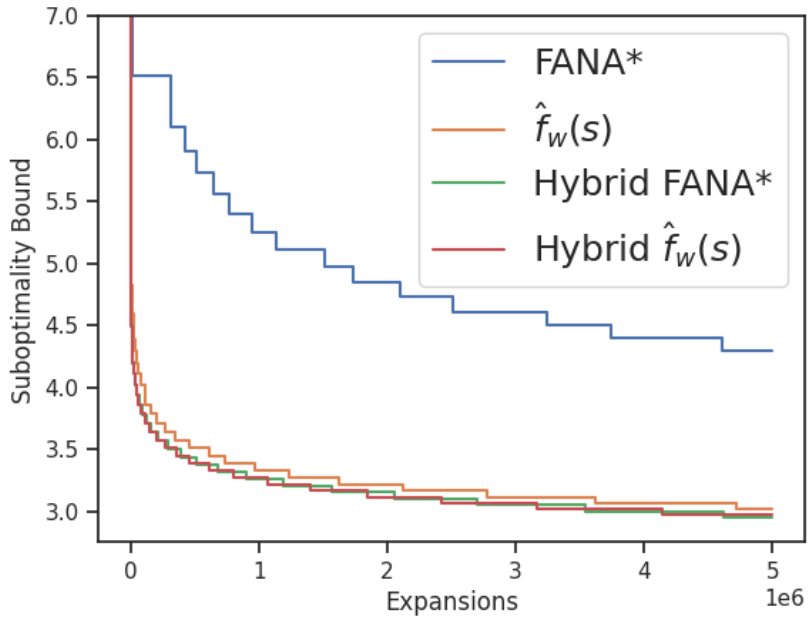


Figure 5.7: The suboptimality bounds per expansion on an instance of the heavy 16-pancake problem.

Non-uniform-cost 20-DoF robot arm. The heuristic used in the robot arm domain does not alter, regardless of the cost of actions or the number of joints. Because of this, it is woefully uninformed in the case of the non-uniform-cost 20-DoF version of the problem. The larger joint count also substantially increases the size of the state space, making the problem much more difficult than the 6-DoF versions. Figure 5.8 shows the suboptimality bounds per expansion achieved by the focal ANA*, $\hat{f}_w(s)$, and their hybrid implementations when limited to 10 million expansions in this domain. The default algorithms perform exceptionally poorly. In particular, $\hat{f}_w(s)$ never improves beyond the suboptimality bound achieved when greedy search discovers the initial solution, and focal ANA* performs little better. The inability to improve beyond the initial solution when using $\hat{f}_w(s)$ is due to the highly uninformed heuristic. The significant difference between the upper and lower bounds causes the algorithm to focus on states that reside in between the two

bounds, but in this domain there are no solutions within the range of f-values that the algorithm is prioritizing. Thus, it never expands a state that leads to an improvement. In contrast to the default implementations, both hybrid algorithms manage to substantially reduce their suboptimality bounds during the search. As is now expected, the hybrid implementation of focal ANA* provides the best suboptimality bounds throughout the search by focusing on intentional improvement of both the upper and lower bounds.

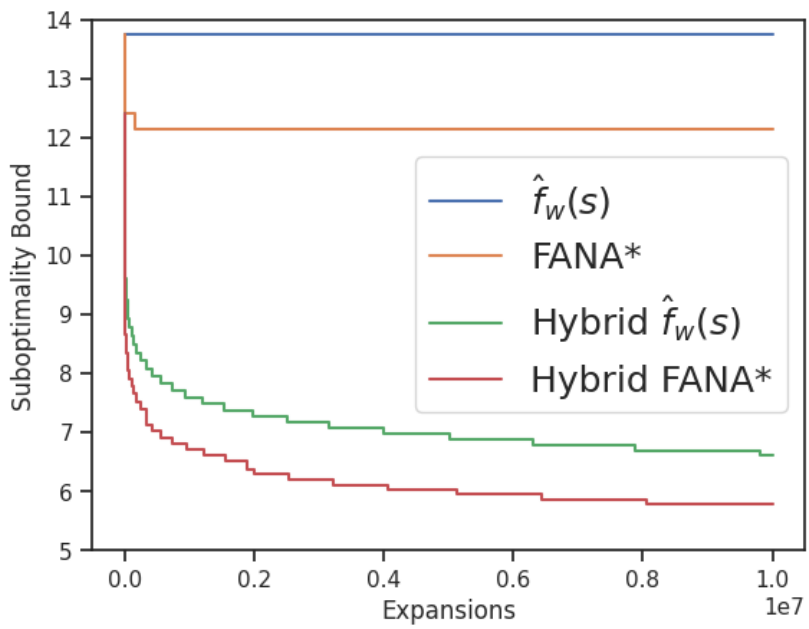


Figure 5.8: The suboptimality bounds per expansion on the non-uniform-cost 20-DoF robot arm.

Hybrid bounded-suboptimal algorithms. The theoretical and practical advantages provided by the hybrid implementation strategy are not limited to anytime algorithms. Bounded-suboptimal algorithms like dynamic potential search guarantee that the first solution discovered is within the desired suboptimality bound, but factors such as the size of the state space, the quality of the

Table 5.4: The number of instances solved for various suboptimality bounds on the heavy 16-pancake problem.

	DPS								
	Default					Hybrid			
Desired ϵ	5.0	4.5	4.0	3.5	3.0	3.0	2.75	2.5	2.25
# Solved	25	23	15	6	1	25	22	7	2

heuristic, or resource limitations may prevent the discovery of any solution. This is due in large part to the fact that the evaluation function will neglect the lower bound, which limits the range of f-values the algorithm can explore in search of a solution. In contrast, a hybrid implementation of a BSS algorithm can periodically raise the lower bound, increasing the range of viable f-values and thus the likelihood of discovering a solution.

Table 5.4 shows the number of instances of the heavy 16-pancake problem that were solvable within 5 million expansions by DPS and its hybrid implementation for several desired suboptimality bounds. The difference in capability is striking; where DPS required a suboptimality bound of 5.0 to successfully solve all 25 instances, the hybrid implementation was able to do so at a suboptimality bound of 3.0. DPS could only solve a single instance with a suboptimality bound of 3.0. Moreover, the hybrid implementation uses half of its budget performing A* expansions, but it is likely that a significant number of these expansions were unable to raise the value of f_{min}, ultimately wasting them. It is possible that a more clever implementation would be capable of achieving even better results by using a more effective budgeting strategy. Ultimately, this hybrid approach to heuristic search provides numerous benefits, both theoretical and practical, and when the evaluation function fits the necessary criteria, the bucket heap facilitates the hybrid strategy while providing significantly faster priority queue operations.

CHAPTER 6

CONCLUSION AND FUTURE RESEARCH

This dissertation has explored theoretical and practical advances of bucket-based priority queues for A* and related bounded-suboptimal and anytime search algorithms. Three new bucket-based priority queue data structures were introduced, and their worst-case asymptotic complexities were analyzed. For the A* algorithm, the lazy two-level bucket queue was shown to have amortized constant complexity for all of its operations—a first for bucket-based priority queues that support the common A* tie-breaking strategy.

The bucket heap data structure extended the applicability of bucket-based priority queues to a large class of bounded-suboptimal and anytime variants of the A* algorithm. The bucket heap was shown to have improved asymptotic complexity compared to a binary heap, and experiments on a wide range of search algorithms and domains showed that use of a bucket heap speeds up search and provides improved suboptimality bounds.

Finally, a hybrid algorithmic strategy that leverages the bucket heap was proposed. This hybrid strategy bounds the worst-case number of state re-expansions performed by bounded-suboptimal and anytime search algorithms that use an inadmissible evaluation function, guaranteeing that the number of state expansions performed by the algorithm is asymptotically equivalent to the number of expansions performed by A*. Experimental results also show that this approach

can provide speedup in many search problems and further improves the suboptimality bounds provided by an algorithm. In the case of bounded-suboptimal algorithms, it was shown that this hybrid approach facilitates the discovery of solutions within desired suboptimality bounds that are otherwise unachievable.

6.1 Future Research

There are several possible directions for future research the stem from the work presented in this dissertation. Many of these relate to either extending the applicability of the bucket heap data structure or leveraging its existing characteristics in additional ways. The hybrid algorithm strategy proposed in Section 5.2 also merits further exploration, given its simplicity and effectiveness.

Real-Valued costs. A limiting factor of bucket-based priority queues is that they are intended to operate in domains that have a small range of integer transition costs. The most straightforward way of dealing with real-valued costs is simply to truncate them into integers and use them directly. This approach is problematic in the context of A* because it removes the guarantee that the solution returned by A* is optimal, although some work has been done in this area [24]. However, the bucket heap can be paired with anytime algorithms, and so the suboptimality that may be introduced by truncating the real-valued costs will eventually converge to 0. Furthermore, if the goal is to find a solution within a desired suboptimality bound, convergence to an optimal solution is not a concern anyway.

Truncating the real-valued costs does still require some additional considerations. The first is that the suboptimality bound $\frac{C}{f_{min}}$ will be loosened by truncation, though it is likely to be insignificant in practice. The second and more difficult to characterize effect is that truncation will

impact an algorithm's expansion order. It is difficult to say to what degree expansion order will be affected, or how much this alteration will affect the performance of the algorithm itself. At minimum, these two questions must be adequately answered before considering this approach to real-valued cost functions viable for the bucket heap.

Bucket aggregation. A second limiting factor of bucket-based priority queues is the range of buckets that are operated on. This is particularly important for two-level bucket structures, including the bucket heap, because inflating the range of primary and secondary buckets increases the constant factor overheads of the priority queue. A possible approach to mitigating these issues is to artificially limit the bucket ranges. Instead of mapping a single priority per bucket, priority values can be grouped, or aggregated, into ranges. The size of these ranges can be a tunable parameter of the structure and a separate parameter could exist for the primary and secondary ranges, providing finer control.

The trade-offs of this approach are difficult to characterize. Firstly, the suboptimality bound is loosened directly by the parameter used to set the primary bucket range. The greater the restriction imposed on the primary bucket count, the worse the suboptimality bound. Additionally, the combination of restrictions on primary and secondary bucket ranges simultaneously affects an algorithm's expansion order in a potentially unpredictable manner. Significant work is needed to explore the advantages and disadvantages of this technique to determine its practical viability, as well as any theoretical ramifications of the change in expansion order.

Parallelization. The inherent structure of bucket-based priority queues may make them useful for parallelization of heuristic graph-search algorithms in multi-threaded environments. There are

two directions that may be promising: parallel expansions and parallel generations. Each poses a slightly different set of advantages and disadvantages. In the two-level bucket queue and structures that extend it, all states residing within a particular secondary bucket are equivalent and can be expanded in any order. A natural approach would be to expand some or all of them simultaneously, in parallel. This necessitates a multi-producer, multi-consumer approach to the priority queue, making appropriate synchronization difficult. Managing dynamic resizing of the underlying bucket structure also becomes more difficult in a parallel environment.

The parallel generation of successors poses slightly fewer problems, though it is likely still difficult. Its usefulness is also more limited because its effectiveness is dependent on the effective branching factor of a domain. However, it requires a multi-producer, single-consumer approach to the priority queue, which is simpler. In both approaches care must be taken to ensure that memory allocations are thread-safe, and careful consideration will be required to avoid performance degradation from false-sharing. The final consideration for both parallel approaches is the hash table. They both require that accessing the hash table is thread-safe and efficient in a multi-threaded environment, further complicating the implementation of a multi-threaded search algorithm.

Hybrid algorithms. The hybrid algorithm strategy proposed in Section 5.2 has strong theoretical and practical implications that merit further exploration. The most obvious question is which evaluation function creates the best hybrid algorithm for each algorithm class? It is possible that no single evaluation function always provides the best results or that a new evaluation function should be created specifically to be used with this hybrid strategy. Answering this question will

require extensive experimentation, and it is likely intertwined with the next open question: what is the best way to alternate expansions in the hybrid strategy?

The implementation discussed in this dissertation used the simplest method of alternating after each expansion. The experimental results showed that for some evaluation functions and domains this is not always effective. The optimal method likely depends on the corresponding evaluation function. In fact, it may be that an evaluation function designed specifically for this hybrid approach, combined with the addition of introspection to the algorithm, can produce a superior strategy.

REFERENCES

[1] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," *Proceedings of the April 18-20, 1967, spring joint computer conference*, 1967, pp. 483–485.

[2] E. Burns, M. Hatem, M. Leighton, and W. Ruml, "Implementing fast heuristic search code," *Proceedings of the International Symposium on Combinatorial Search*, 2012, vol. 3, pp. 25–32, Issue: 1.

[3] H. Carrillo and D. Lipman, "The multiple sequence alignment problem in biology," *SIAM journal on applied mathematics*, vol. 48, no. 5, 1988, pp. 1073–1082.

[4] L. Cohen, M. Greco, H. Ma, C. Hernández, A. Felner, T. S. Kumar, and S. Koenig, "Anytime Focal Search with Applications.," *IJCAI*, 2018, pp. 1434–1441.

[5] T. Cormen, C. Leiserson, R. Rivest, and C. Stein, *Introduction to Algorithms*, 4 edition, MIT Press, 2022.

[6] R. B. Dial, "Algorithm 360: shortest-path forest with topological ordering [H]," *Communications of the ACM*, vol. 12, no. 11, Nov. 1969, pp. 632–633.

[7] E. W. Dijkstra, "A Note on Two Problems in Connexion With Graphs," *Numerische Mathematik*, vol. 1, no. 1, 1959, pp. 269–271.

[8] W. H. Gates and C. H. Papadimitriou, "Bounds for sorting by prefix reversal," *Discrete mathematics*, vol. 27, no. 1, 1979, pp. 47–57.

[9] D. Gilon, A. Felner, and R. Stern, "Dynamic potential search—a new bounded suboptimal search," *Proceedings of the International Symposium on Combinatorial Search*, 2016, vol. 7, pp. 36–44, Issue: 1.

[10] D. Gilon, A. Felner, and R. Stern, "Dynamic potential search on weighted graphs," *Proceedings of the International Symposium on Combinatorial Search*, 2017, vol. 8, pp. 119–123.

[11] E. Hansen, S. Zilberstein, and V. Danilchenko, "Anytime heuristic search: First results," *Univ. Massachusetts, Amherst, MA, Tech. Rep*, vol. 50, 1997.

[12] E. A. Hansen and R. Zhou, "Anytime heuristic search," *Journal of Artificial Intelligence Research*, vol. 28, no. 1, Jan. 2007, pp. 267–297, ARXIV_ID: 1110.2737 MAG ID: 1622225657 S2ID: aa85b00aada6f2351a51b16ea5b85b9652ea47e8.

[13] P. E. Hart, N. J. Nilsson, and B. Raphael, "A Formal Basis for the Heuristic Determination of Minimum Cost Paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, July 1968, pp. 100–107, MAG ID: 1969483458.

[14] M. Helmert, "Landmark heuristics for the pancake problem," *Proceedings of the International Symposium on Combinatorial Search*, 2010, vol. 1, pp. 109–110.

[15] T. Ikeda and H. Imai, "Fast a algorithms for multiple sequence alignment," *Genome Informatics*, vol. 5, 1994, pp. 90–99.

[16] T. Ikeda and H. Imai, "Enhanced A algorithms for multiple alignments: optimal alignments for several sequences and k-opt approximate alignments for large cases," *Theoretical Computer Science*, vol. 210, no. 2, 1999, pp. 341–374.

[17] C. L. Jackins and S. L. Tanimoto, "Quad-trees, Oct-trees, and K-trees: A generalized approach to recursive decomposition of euclidean space," *IEEE transactions on pattern analysis and machine intelligence*, , no. 5, 1983, pp. 533–539.

[18] R. E. Korf, "Depth-first iterative-deepening: An optimal admissible tree search," *Artificial intelligence*, vol. 27, no. 1, 1985, pp. 97–109, Publisher: Elsevier.

[19] M. Likhachev, D. Ferguson, G. Gordon, A. Stentz, and S. Thrun, "Anytime search in dynamic graphs," *Artificial Intelligence*, vol. 172, no. 14, Sept. 2008, pp. 1613–1643, MAG ID: 2028945152.

[20] M. Likhachev, G. Gordon, and S. Thrun, "ARA*: Anytime A* with Provable Bounds on Sub-Optimality," *Proc. of the 16th International Conference on Neural Information Processing Systems*, Cambridge, MA, USA, 2003, pp. 767–774, MIT Press.

[21] A. Martelli, "On the complexity of admissible search algorithms," *Artificial Intelligence*, vol. 8, no. 1, 1977, pp. 1–13.

[22] D. Meister, S. Ogaki, C. Benthin, M. J. Doyle, M. Guthe, and J. Bittner, "A survey on bounding volume hierarchies for ray tracing," *Computer Graphics Forum*, vol. 40, no. 2, 2021, pp. 683–712.

[23] J. Pearl and J. H. Kim, "Studies in semi-admissible heuristics," *IEEE transactions on pattern analysis and machine intelligence*, , no. 4, 1982, pp. 392–399.

[24] J. Ros-Giralt, A. Commike, P. Cullen, and R. Lethin, "Accelerating Dijkstra's Algorithm Using Multiresolution Priority Queues," *Proc. of the 22nd Annual IEEE Conference on High Performance Extreme Computing*, 2018, pp. 1–7.

[25] R. Stern, A. Felner, J. Van Den Berg, R. Puzis, R. Shah, and K. Goldberg, "Potential-based bounded-cost search and Anytime Non-Parametric A∗," *Artificial Intelligence*, vol. 214, 2014, pp. 1–25, Publisher: Elsevier.

[26] R. E. Tarjan, "Amortized computational complexity," *SIAM Journal on Algebraic Discrete Methods*, vol. 6, no. 2, 1985, pp. 306–318.

[27] J. Thayer, J. Benton, and M. Helmert, "Better parameter-free anytime search by minimizing time between solutions," *Proceedings of the International Symposium on Combinatorial Search*, 2012, vol. 3, pp. 120–128.

[28] J. T. Thayer and W. Ruml, "Faster than Weighted A*: An Optimistic Approach to Bounded Suboptimal Search.," *ICAPS*, 2008, pp. 355–362.

[29] J. van den Berg, R. Shah, A. Huang, and K. Goldberg, "ANA*: Anytime Nonparametric A*," *Proc. of the 25th AAAI Conference on Artificial Intelligence*, 2011, pp. 105–111.

[30] J. Williams, "Heapsort," *Commun. ACM*, vol. 7, no. 6, 1964, pp. 347–348.

[31] T. Yoshizumi, T. Miura, and T. Ishida, "A* with Partial Expansion for Large Branching Factor Problems.," *AAAI/IAAI*, 2000, pp. 923–929.

[32] R. Zhou and E. Hansen, "Multiple Sequence Alignment Using Anytime A*.," *Proc. of the 18th AAAI Conference on Artificial Intelligence*, 2002, pp. 975–977.

[33] R. Zhou and E. A. Hansen, "Sweep A: space-efficient heuristic search in partially ordered graphs," *Proceedings. 15th IEEE International Conference on Tools with Artificial Intelligence*. IEEE, 2003, pp. 427–434.